PyEXPINT: A Validated Backend-Independent Framework for Exponential General Linear Integrators

FRANCISCO R. VILLATORO, Escuela de Ingenierías Industriales, Universidad de Málaga, Spain

Exponential integrators treat a dominant linear operator exactly and the remaining nonlinear dynamics explicitly, but a usable implementation must also fix how method coefficients, matrix-function actions, history, adaptivity and diagnostics interact. PyEXPINT is a Python framework that expresses exponential Runge–Kutta, multistep and general linear schemes in a single explicit exponential general linear form and executes the same method definitions through interchangeable realizations of the action of phi-functions. The release reconstructs the 47-entry catalogue of the EXPINT package. The archived cross-check of two specified problems reports near-roundoff agreement with the original 2005 implementation for 42 entries, agreement after a one-line correction for four further entries, and regenerated output coefficients for the remaining entry. The framework provides an operator-expression layer for combinations of phi-functions, dense, diagonal, full-Arnoldi, KIOPS-type, real-Leja and hybrid realizations, a rule-based selector, algebraic fusion of compatible actions, fixed-step integration with explicit startup policies, step-doubling control and the embedded ETD34 pair, whose error estimate is shown to vanish for state-independent forcing. Work-precision experiments show that method rankings depend on the work observable, and paired timing campaigns show that backend conclusions are configuration and hardware dependent, so algorithmic work and achieved accuracy are reported as primary evidence. The manuscript package preserves comparison records and data-to-figure and data-to-table transformations; the software candidate and its execution-level validation are identified separately.

CCS Concepts: • **Mathematics of computing** → **Numerical analysis**; • **Software and its engineering** → *Software libraries and repositories*; • **Computing methodologies** → *Modeling and simulation.*

Additional Key Words and Phrases: exponential integrators, general linear methods, matrix functions, Krylov methods, KIOPS, Leja interpolation, scientific software, reproducible benchmarking

ACM Reference Format:
Francisco R. Villatoro. 2026. PyEXPINT: A Validated Backend-Independent Framework for Exponential General Linear Integrators. *ACM Trans. Math. Softw.* 1, 1 (October 2026), 33 pages.

1 Introduction

For a semilinear initial-value problem

$$y'(t) = Ly(t) + N(t, y(t)), \tag{1}$$

exponential integrators incorporate the linear flow generated by L into the time-discretization coefficients. This idea supports high-order methods for stiff and highly oscillatory problems while replacing repeated linear solves by actions of the exponential and the related φ functions [12]. The numerical method and the algorithm used to evaluate those actions are conceptually distinct, but software often entangles them. That coupling becomes increasingly restrictive when the same

Author's Contact Information: Francisco R. Villatoro, Escuela de Ingenierías Industriales, Universidad de Málaga, Málaga, Spain, frvillatoro@uma.es.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7295/2026/10-ART
https://doi.org/

time integrator should be used with a diagonal spectrum, a dense reference matrix, a large sparse operator or a matrix-free linear map.

EXPINT was an early software response to this problem. Its MATLAB catalogue made exponential Runge–Kutta, multistep and general linear integrators available in a common package and emphasized rapid deployment and testing [3]. Since then, matrix-function-vector algorithms have evolved substantially. Krylov algorithms for combinations of φ functions, from Expokit to Algorithm 919 and KIOPS [9, 17, 22], avoid materializing matrix functions. Polynomial methods based on Leja interpolation provide a different tradeoff when usable spectral information is available [4, 5, 7]. General-purpose scientific libraries also provide scalable exponential actions such as `expm_multiply` [1, 24]. These developments motivate a software boundary in which a time integrator requests matrix-function actions without being rewritten for each numerical realization. The boundary itself is not new: Expokit and Algorithm 919 are reusable action routines, and OrdinaryDiffEq.jl switches between cached matrix functions and Krylov actions with one option. What is new here is what is built on it.

PyEXPINT is an independent Python framework built around that boundary. Its design is not a literal translation of the historical EXPINT source. The historical catalogue is reconstructed as mathematical data in a single `MethodSpec` representation; the execution engine interprets those data; and action backends implement the numerical realization. This separation allows the same method specification to operate on dense, diagonal, sparse and matrix-free problems and makes it possible to instrument work at the matrix-function layer. It also exposes a useful verification surface: method metadata, coefficient identities, history layout, action algebra and backend statistics can be tested separately rather than being hidden inside a method-specific solver.

The software contribution is therefore broader than "EXPINT in Python" but narrower than a claim to a new universally superior exponential solver. The article makes five main contributions.

(1) We encode a 47-entry historical catalogue in a common explicit exponential general linear method (EGLM) representation, with separate metadata for classical, strong stiff and weak or conditional order.
(2) We introduce an analytic operator-expression layer and a backend protocol that decouple method coefficients from matrix-function-action algorithms. Compatible φ actions can be fused automatically, including sums with distinct right-hand sides when the backend supports them, and external backends enter through one hook.
(3) We provide fixed-step execution with explicit startup policies for history methods, generic step-doubling adaptivity and the embedded ETD34 pair with action-tolerance coupling, failure rejection and explicit termination rules, and we document a limitation of the published ETD34 estimator, which vanishes for state-independent forcing.
(4) We combine structural and coefficient checks with a numerical cross-check against the original EXPINT implementation. The supplied summary reports near-roundoff agreement for 42 of the 47 entries, agreement with a one-line correction for four entries, and regenerated output coefficients for one entry. The summary and its limitations are made explicit in Section 4.3.
(5) We supply work instrumentation and reproducibility material that separates portable quantities, namely error, matrix-vector products, nonlinear evaluations, accepted and rejected steps and Krylov or polynomial diagnostics, from machine-dependent elapsed time. Historical runs, author-reported candidate runs and manuscript postprocessing are distinguished in the experimental record. Timing results are retained as paired observations. A broadband problem without manufactured forcing is also included, with its original numerical record, reference state and individual timing repetitions retained in the private companion.

This scope determines what the article does not claim. No runtime superiority over external packages such as EXPODE, LeXInt, `rkstiff` or the reference KIOPS implementation is asserted; the relationship to them is functional (Section 8). No distributed-memory scalability claim is made because the release has no distributed backend interface. The rule-based automatic backend is presented as configurable and problem-dependent, not as a globally optimal selector; on the canonical timing problem it chooses the slower backend. And the default numerical startup does not in general preserve the nominal orders six and seven of the high-order history methods.

The remainder of the paper follows the software boundary itself. Section 2 states the common mathematical representation and action algebra. Section 3 describes external state, startup, execution, work accounting and the release-facing API. Section 4 presents the validated historical catalogue, its reconciliations and the cross-check against the original sources. Section 5 describes the action backends and fusion contract, and Section 6 the adaptive interfaces. The numerical program is reported in Section 7. Related software is positioned in Section 8; reproducibility, AI-assisted research use and limitations are discussed in Section 9; and Section 10 concludes.

2 Mathematical framework

2.1 Semilinear problems and φ functions

We consider finite-dimensional semidiscretizations of the form

$$y'(t) = Ly(t) + N(t, y(t)), \qquad y(t_0) = y_0, \tag{2}$$

where L is a linear operator and N contains the remaining dynamics. The implementation does not require L to be stored as a dense matrix: depending on the backend, it may be diagonal data, a dense or sparse matrix, or an object that supplies matrix-vector products.

The analytic coefficient functions are expressed through

$$\varphi_0(z) = e^z, \qquad \varphi_k(z) = \int_0^1 e^{(1-\theta)z} \frac{\theta^{k-1}}{(k-1)!}\, d\theta, \quad k \geq 1. \tag{3}$$

A method never needs a materialized matrix $\varphi_k(hL)$ as part of the public execution contract. It needs the *action* of an analytic coefficient, often a linear combination of several $\varphi_k(chL)$, on one or more vectors. This distinction is central for large-scale problems and for comparing Krylov and polynomial realizations.

2.2 One explicit exponential GLM representation

Let $X_n = (X_{n,1}, \ldots, X_{n,r})^T$ denote the external state at time t_n. The first component is the physical numerical solution. For the historical multistep methods represented in PyEXPINT, the remaining components store scaled nonlinear history. The representation is the exponential general linear form of Berland, Skaflestad and Wright [3], with the order theory of Ostermann, Thalhammer and Wright [18] and the B-series order conditions of Berland, Owren and Skaflestad [2] behind the metadata; exponential Runge–Kutta schemes such as those of Cox and Matthews [6] and Krogstad [15], Lawson's integrating-factor methods [16] and the exponential Adams methods of Nørsett and of Hochbruck and Ostermann [13] are its special cases. A `MethodSpec` contains nodes c, analytic operator-coefficient arrays U, A, V, B, and order/provenance metadata. With

$$G_j = hN(t_n + c_j h, Y_j), \tag{4}$$

the common explicit engine evaluates

$$Y_i = \sum_{j=1}^{r} U_{ij}(hL) X_{n,j} + \sum_{j<i} A_{ij}(hL) G_j, \tag{5}$$

$$X_{n+1,i} = \sum_{j=1}^{r} V_{ij}(hL) X_{n,j} + \sum_{j=1}^{s} B_{ij}(hL) G_j. \tag{6}$$

The strict lower-triangular condition on A encodes explicit stage ordering. For $r > 1$ the catalogue uses a nonlinear-history layout: after the physical output row, the engine inserts $hN(t_n, y_n)$ and shifts older history. This makes multistep and general-linear schemes an external-state specialization of the same step function rather than separate solver families.

The representation is deliberately descriptive. Besides the coefficient arrays, a method records its family, classical order, stiff order when known, weak or conditional stiff order when distinct, stage order, quadrature order, number of nonlinear evaluations, references, historical filename, notes and a provenance status. Consequently an executable method need not collapse classical, stiff and weak-order statements into one integer.

2.3 Order metadata are not interchangeable

For nonstiff problems, the usual classical convergence order describes the asymptotic temporal error under appropriate smoothness. In semilinear stiff problems, stronger operator-valued order conditions can determine whether the same order is retained uniformly; failure of those conditions can produce order reduction [11, 12]. Several historical affine Lie-group entries additionally satisfy useful conditions at $z = 0$ without satisfying the corresponding strong identity for general z. PyEXPINT stores these as separate strong and weak or conditional metadata rather than resolving a textual discrepancy by choosing the larger label.

The distinction matters experimentally. A finite-grid manufactured problem may show a higher apparent slope because a leading defect vanishes for that problem. Such a slope is useful validation evidence but does not by itself change the catalogue's algebraic order metadata. Section 7 therefore displays both ordinary classical tests and semidiscrete stiff-order diagnostics.

2.4 Analytic operator expressions

Method coefficients are represented as analytic expression trees rather than backend-specific arrays. The basic object

$$C(z) = \sum_{\ell} \alpha_\ell \varphi_{k_\ell}(c_\ell z) \tag{7}$$

is a `PhiCombination`. Sums of such expressions are canonicalized so that a backend with native combination support can construct one numerical action rather than evaluate every term independently. Products represent composition of analytic operator coefficients. A shifted-resolvent expression is kept separate from the φ algebra and is used only by the retained historical Crank–Nicolson comparator.

This symbolic layer is small but consequential. It allows the catalogue to state *what* operator function a coefficient represents, while the backend decides *how* its action is approximated. It is also the point at which algebraic fusion becomes a semantics-level transformation rather than a method-specific optimization.

3 Software architecture and execution model

Figure 1 summarizes the central design. A `MethodSpec` and its analytic coefficient algebra are interpreted by one EGLM engine. Numerical φ and exponential actions are delegated through

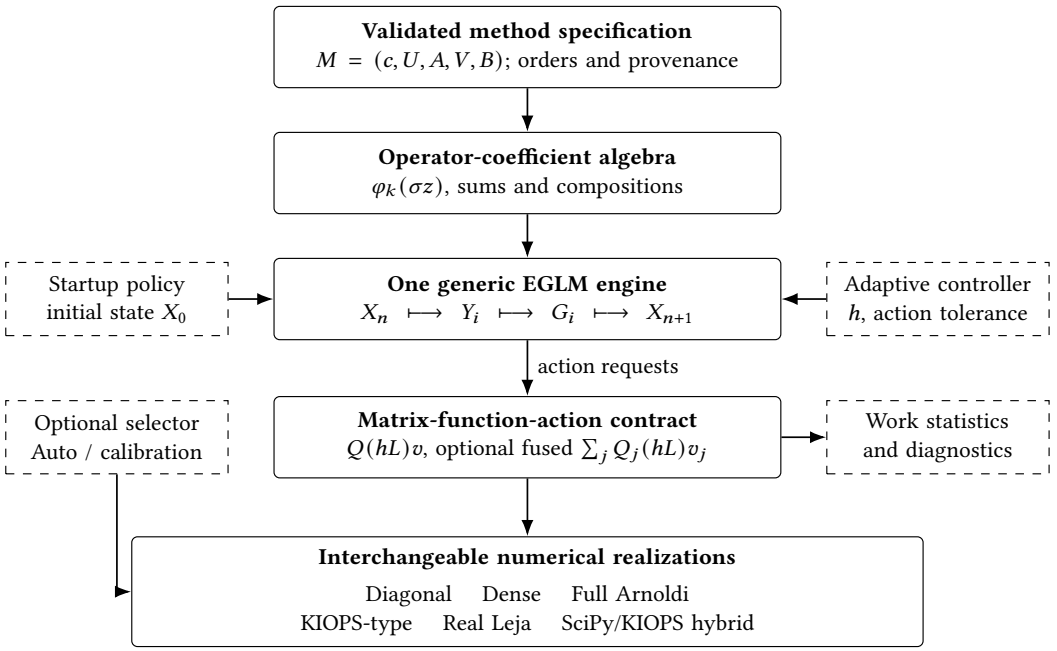


Fig. 1. Release architecture of PyEXPINT. Mathematical method definitions enter a common operator-coefficient algebra and explicit EGLM engine. The engine requests actions from interchangeable numerical backends, while startup, adaptive control, dispatch and statistics remain separate layers.

a backend boundary. Startup, adaptivity, backend selection and instrumentation are orthogonal services around that path; they are not embedded in 47 separate method implementations.

3.1 Problem, state and solution objects

A `SemilinearProblem` contains L, $N(t, y)$, the initial vector, a time interval and, optionally, an exact solution used for validation or exact startup. Its minimal contract intentionally does not impose a PDE mesh, Jacobian representation or spectral decomposition. An `ExternalState` stores the time and the $r \times n$ external-state array required by the chosen GLM; its first row is always the physical state. A `Solution` stores the sampled times and states together with the method and backend identifiers, step metadata and a statistics mapping.

This separation prevents a common source of hidden coupling. A method does not own an L representation, and a backend does not own the nonlinear model. The only place where both meet is the action context created for a particular step size.

3.2 The common step function

The implementation of `step_external` follows Eqs. (5)–(6) directly. At each stage it first forms the action of the U row on the external state and, for later stages, the action of the strict-lower A row on previously computed nonlinear vectors. It evaluates N once at the stage node, stores hN, and then forms the external outputs from V and B.

The action-sum helper first asks the bound backend for an optional native fused-sum operation. If that operation is not available, it falls back to individual analytic-expression actions. Thus backend-specific fusion is optional without changing method semantics. This same mechanism is responsible

for the controlled fusion ablation in Section 7.2, where the unfused path of a backend is obtained by hiding the fused operation from its bound context.

3.3 History and startup

A one-step method has $r = 1$ and needs no history. For a method whose external layout stores nonlinear history, $r - 1$ previous nonlinear values must be available before the main method can advance. PyEXPINT makes that initialization policy explicit. `ExactStartup` uses the problem's exact solution and is intended primarily for validation. `OneStepStartup` uses any one-step `MethodSpec`; HochOst4 is the default numerical starter when no exact solution is supplied.

The numerical starter bounds the accuracy a history method can reach in general. Each startup interval is advanced with a single step of the starter, so the starting values carry its local error $O(h^{q+1})$, q the order of the starter. Under the usual regularity and stability assumptions, and for a fixed spatial discretization, the end-to-end error of a method of order p is then bounded by $C_1 h^p + C_2 h^{q+1}$. This provides an order guarantee $\min(p, q + 1)$, hence $\min(p, 5)$ with HochOst4; it does not impose a universal upper bound on an observed order when leading terms cancel. Section 7.7 measures this for the order six and seven entries of the catalogue. Making startup visible therefore matters both for accuracy and for work comparisons: a benchmark that uses exact history isolates the asymptotic method body, and it must not present that cost, or that order, as a complete practical solve. This distinction is stated explicitly for the fixed-window work–precision experiment in Section 7.3.

3.4 Work accounting

A backend that supplies a matrix-vector counter reports its cumulative value in the top-level `matvecs` entry of its statistics. Selectors that delegate to several engines report the sum there and keep per-engine breakdowns in nested entries, which the adaptive solvers never add to the total. The work of an attempt is the increment of that counter; when a backend reports no counter, for example an exact diagonalization, the work is recorded as missing rather than replaced by another quantity. Nonlinear evaluations are counted by wrapping the user's function, so that failed attempts are included. These rules are what make the work observables of Section 7 comparable across backends.

3.5 Interface usage fragment

The following fragment illustrates the solver interface once a semilinear `problem` and the listed PyEXPINT objects have been imported or constructed. It is not a standalone program: the spatial grid, initial state and imports belong to the complete example in the software companion. No exact solution is supplied for this use of numerical startup.

```
reference = solve_fixed(
    problem, HOCHOST4, 2/512,
    KiopsBackend(tol=1e-12)).y[-1]
multistep = solve_fixed(
    problem, ABNORSETT4, 2/128,
    KiopsBackend(tol=1e-11))
adaptive = solve_etd34(
    problem, KiopsBackend(tol=1e-11),
    options=ETD34Options(rtol=1e-8, atol=1e-10))
print(adaptive.stats)
```

The reference here is a numerical comparison, not an exact solution or a certified error bound. The complete program is supplied as `reproducibility/no_exact_history.py`, byte-identical to `software/examples/no_exact_history.py` in the private companion. Its clean-installation log records a comparison difference of 6.53×10^{-9} for ABNorsett4 and 7.39×10^{-9} for ETD34, with 56 accepted and one rejected adaptive step, in the recorded environment. These are example-validation outputs, not a performance benchmark.

A method is added by constructing a `MethodSpec` from its nodes and analytic coefficient arrays. A backend provides `bind(L,h)`, returning an action context, and `stats()`, with an authoritative top-level `matvecs` total when that metric is available. The adaptive interface also requests `with_tolerance(tol)`. A callable hook takes precedence over the built-in cloning cases and is responsible for preserving the required non-tolerance configuration. Independent mutable statistics are needed when a new stateful work backend is returned; the supplied protocol also permits a stateless realization to return itself. Each solve resets available work counters. The supplied `test_c4_cloning_and_startup.py` covers a minimal external realization and preservation of the calibrated selector's type, record and thresholds. Unsupported external backends without the hook are rejected; built-in backends have explicit cloning branches. Fixed-step execution uses the bound action context directly.

3.6 Release-facing API

The release distinguishes public availability from structural stability. The stable core includes problem, solution and state objects; `MethodSpec` and catalogue accessors; fixed, generic adaptive and ETD34 solvers; startup policies; basic analytic-expression constructors; the principal action backends; calibrated selection; and workspace estimation. External backends enter the adaptive solvers through the `with_tolerance` hook described in Section 6. Internal expression tree classes, the backend-statistics dictionary beyond the `matvecs` entry, and selector serialization remain public for research convenience but are not structurally frozen. Variable-step multistep and general linear methods and distributed interfaces are explicitly outside the compatibility contract.

This release boundary is relevant to reproducibility: benchmark scripts may inspect richer statistics, but the paper does not equate those internal keys with a permanent user API.

4 Validated reconstruction of the historical catalogue

4.1 Scope and provenance

The release contains 47 catalogue entries associated with the historical EXPINT method set [3], including Runge–Kutta, Lawson, exponential Adams, predictor–corrector and general linear, affine Lie-group and generalized Lawson schemes, and the retained Crank–Nicolson comparator. The full metadata table appears in Supplementary Table S1. The objective of the reconstruction was not to copy the historical MATLAB routines line for line, but to encode the method intended by the published construction and to retain enough provenance to explain every canonical decision.

For families that can be generated from interpolation or moment conditions, the construction uses exact rational arithmetic for polynomial coefficients and converts only the final scalar multipliers of analytic operator expressions to floating point. This reduces the transcription surface and makes neighbouring members of a family share one derivation. The generalized and modified Lawson families of Krogstad [15] and Ostermann, Thalhammer and Wright [18], for example, are generated from one construction rather than maintained as ten unrelated coefficient tables.

Table 1. Canonical coefficient choices recorded for the five differing entries. E identifies the four cases for which the supplied comparison summary reports agreement after a one-line correction; R identifies regenerated output coefficients. The numerical differences are reproduced in Supplementary Table S6. Source hashes, correction diffs and refinement outputs are archived separately with the summary. The coefficient contrasts in this table are the reconstruction's recorded choices, not a new execution of the historical sources.

Entry		Recorded coefficient contrast	Supplied numerical comparison
ETD2RK	E	Recorded original $b_1 = \varphi_1 - 2\varphi_2$, $b_2 = \varphi_2$; canonical $b_1 = \varphi_1 - \varphi_2$.	Original scalar/matrix differences $2.5 \times 10^{-1}/2.8 \times 10^{-1}$; corrected $0/5.0 \times 10^{-15}$.
ETD2CF3	E	Recorded original $a_{21} = \frac{1}{3}\varphi_1(z/2)$ at $c_2 = 1/3$; canonical $a_{21} = \frac{1}{3}\varphi_1(z/3)$.	Original $3.4 \times 10^{-7}/5.3 \times 10^{-7}$; corrected $2.7 \times 10^{-15}/1.7 \times 10^{-15}$.
ETD5RKF	E	Recorded scalar-branch $U_{41} = e^{3z/5}$; canonical $U_{41} = e^{3z/4}$, also recorded for the matrix branch.	Original $6.5 \times 10^{-3}/8.5 \times 10^{-15}$; corrected $2.3 \times 10^{-14}/8.5 \times 10^{-15}$.
ABLawson2	E	Recorded history weight $-\frac{1}{2}e^{z/2}$; canonical $-\frac{1}{2}e^{2z}$.	Original $3.6 \times 10^{-2}/3.7 \times 10^{-2}$; corrected $2.3 \times 10^{-16}/6.2 \times 10^{-15}$.
ModGenLawson45	R	Output weights regenerated from the published quadrature construction; six differing values are listed at $z = -0.3$.	Original scalar/matrix differences $7.7 \times 10^{-9}/2.5 \times 10^{-8}$; no corrected final-state comparison is supplied.

4.2 Verification hierarchy

A historical conflict was not resolved by choosing whichever version produced the most attractive convergence plot. The audit used a hierarchy of evidence: structural dimensions and explicitness; coefficient identities and zero-operator limits; published stage nodes and quadrature constructions; strong and weak operator-valued order conditions when available; numerical regressions; and, finally, execution of the original MATLAB files. The resulting record separates an executable discrepancy from a metadata or semantic discrepancy. Table 1 lists the five executable or regenerated cases and Table 2 the six metadata cases retained in the main paper.

4.3 Cross-check against the original sources

The private companion supplies the drivers, archived numerical outputs and summary of the comparison with the 2005 EXPINT files. The original MATLAB sources are not redistributed; seven source-file SHA-256 values identify the five discrepant scheme files and the two common routines listed in the cross-check record. The recorded interpreter is GNU Octave 8.4.0. The script `software/tools/expint_crosscheck/crosscheck_expint.m` specifies both problems: $y' = -y + y^2$, $y(0) = 0.4$, with exact startup, and a two-dimensional system with

$$L = \begin{pmatrix} -1 & 0.3 \\ 0.2 & -2 \end{pmatrix}, \qquad N(y) = y^2, \qquad y_0 = (0.4, 0.3)^T,$$

where squaring is componentwise and startup is numerical. Both use $[0, 1.6]$ and $h = 0.1$. The CSV contains 102 output rows: two problems for 47 original schemes and four corrected variants. The Python comparison driver specifies the matching PyEXPINT computations and its 10^{-12} comparison criterion.

The archived comparison reports agreement within that criterion on both problems for 42 entries, with largest relative differences 1.3×10^{-15} and 5.9×10^{-15}. ABLawson2, ETD2RK, ETD5RKF and ETD2CF3 agree after the four documented one-line corrections; the largest of their eight reported

Table 2. Metadata and semantic reconciliations (M): the executable formula does not change; the recorded order metadata and their interpretation do.

Entry		Historical label	Recorded metadata	Evidence
EGLM332	M	Header labels $p = 4, s = 2, r = 3$.	Dimensions give $s = 3, r = 2$; audited $Q = 2, P = 4$ gives recorded $p = 3$.	Structural and order-condition checks.
PEC726	M	Header labels order 6 and $r = 5$.	$p = 7, s = 2, r = 6$; interpolation and moment conditions agree.	Stage and quadrature conditions through the declared order.
PEC423	M	README labels stiff order 3.	Recorded stiff order 4 from $Q = 3, P = 4$.	Order conditions of [18].
PECEC433	M	README labels stiff order 3.	Recorded stiff order 4 from $Q = 3, P = 4$.	Order conditions of [18].
RKMK2e	M	Source and README order labels 2 and 1.	Strong stiff order 1; weak order 2. The strong second moment fails away from $z = 0$.	Strong and weak moment test; stiff-problem corroboration.
RKMK4t	M	Source and README order labels 1 and 2.	Same strong and weak distinction; no coefficient change follows.	Strong and weak moment test; stiff-problem corroboration.

corrected differences is 2.3×10^{-14}. ETD5RKF already agrees on the matrix problem to 8.5×10^{-15} while its uncorrected scalar difference is 6.5×10^{-3}. Unified diffs of the four changes accompany the outputs. ModGenLawson45 remains distinct: its output coefficients are regenerated from the published construction. Six differing coefficient pairs at $z = -0.3$ are tabulated, but no agreement of a corrected final-state implementation is inferred for that entry.

Supplementary Table S6 reproduces the unchanged summary. Its coefficient indices are one-based, as specified by the supplied drivers. Refinement outputs and an ETD5RKF storage-representation comparison are also archived separately. These numerical comparisons support the stated reconstruction; they are neither new convergence theorems nor certification of all inputs admitted by the software.

4.4 Catalogue-wide regression

The private validation records contain 275 passing tests on the source snapshot and the same 275 test identifiers against the installed wheel, under Python 3.11.15, NumPy 1.24.3, SciPy 1.10.1 and pytest 7.1.2. Of these, 80 exercise the C2–C4 work, adaptive and cloning contracts; the supplied negative-control log records 53 failures and 27 passes when those tests are applied to the earlier C1 baseline. These logs are from the preparation environment; they are not cluster timing records. The repository contains a Python 3.11–3.14 CI configuration, but the supplied candidate logs do not establish execution of the entire version matrix.

The evidence map `reproducibility/coverage/catalogue_test_map.csv` connects all 47 entries to source definitions, structural validation, named scalar-order regressions and the linear tests, and lists dedicated coefficient or moment tests where they exist. Its README identifies the JUnit record and the scope of each evidence category. Moment residuals evaluated at selected

arguments are numerical checks, not proofs of a functional identity. The scalar regression accepts the best eligible adjacent slope; it is not a universal convergence result. Additional symbolic constructions are present in the source, but no execution of the symbolic audit is inferred from the pytest log.

Supplementary Table S1 records the coefficient provenance, order metadata and cross-check classification. The evidence map complements that table without promoting metadata or the total number of tests into a proof of all stiff-order assertions.

4.5 What "validated" means here

Throughout this article, *validated catalogue* means that the release has a traceable canonical specification, machine-checkable structural and algebraic checks where implemented, numerical regressions consistent with the recorded interpretation, and agreement to roundoff with the original implementation wherever the two are meant to coincide. It does not mean formal verification in a proof assistant, nor does it certify every stiff-order assertion for every infinite-dimensional problem class. This narrower meaning is the one relevant to a software artifact intended to preserve and execute a heterogeneous historical method collection.

5 Matrix-function-action backends

The common engine reduces a time step to analytic operator actions. A backend implements those actions for a particular representation of L and may expose additional operations such as a native combination action, a fused sum across distinct right-hand sides, or a shifted solve. Table 3 summarizes the release realizations and the scope of their work diagnostics.

5.1 Reference realizations

DiagonalBackend evaluates each analytic function on supplied diagonal entries and is useful when the linear operator is already diagonalized. It is used in the stiff-order experiments to remove iterative matrix-action error from temporal refinement. **DenseBackend** materializes small dense matrix-function calculations and serves as a reference path; it is not intended as the large-scale default.

A **full-Arnoldi** backend constructs an orthonormal Krylov basis and approximates the action through the projected problem. Its role is both numerical and diagnostic: because it retains full orthogonalization, it gives a transparent reference against which the more specialized Krylov backend and the fusion transformation can be exercised.

5.2 KIOPS-type actions

The KIOPS-type backend follows the augmented-Krylov philosophy of evaluating linear combinations of φ functions through a projected exponential, with adaptive substepping and incomplete orthogonalization [9]. It is an independent implementation from the published algorithmic description; the reference KIOPS MATLAB source is not vendored or translated, which is why the backend is not called KIOPS. It records matrix-vector products, accepted and rejected internal substeps, projected dimensions and action failures. These diagnostics are used as algorithmic work observables, not as a claim that one product has identical wall cost across all backends. Algorithm 919 is an important predecessor for this action-centric view: a polynomial inhomogeneous linear problem can be recast as the action of a combination of the exponential and φ functions, precisely the operation required repeatedly by exponential integrators [17].

Table 3. Numerical realizations of the common action contract. Workspace estimates concern dominant algorithmic storage for fixed maximum φ-index, excluding the problem, sparse matrix, Python objects and uncounted library temporaries. Auto is a dispatch policy, not another matrix-function algorithm.

Realization	Operator form	Spectral information	Distinct-RHS fusion	Dominant workspace	Work observable
Diagonal	Given diagonal	Eigenvalues supplied	Trivial	$O(n)$	Direct scalar-function work.
Dense	Materialized matrix	None required	Limited	$O(n^2)$	Dense augmented exponentials; reference, not exact arithmetic.
Full Arnoldi	Sparse or matrix-free	None required	Yes	$O(nm + m^2)$	Matvecs, orthogonalization and projected work.
KIOPS-type	Sparse or matrix-free	None required	Yes	$O(nm + m^2)$	Matvecs, incomplete orthogonalization and adaptive substeps.
Real Leja	Sparse or matrix-free	Reliable real interval	No, currently	$O(n + d)$	Polynomial recurrence and stopping tests.
SciPy/KIOPS hybrid	Sparse or matrix-free	None required	Partial	Realization-dependent	KIOPS counts omit internal SciPy matvecs.

For one projection, full Arnoldi adds $O(nm^2)$ orthogonalization work; incomplete orthogonalization of fixed length ℓ adds $O(\ell nm)$. Small projected matrix-function work is additional. A Leja recurrence of degree d requires polynomial matvecs and $O(nd)$ vector work. Substeps and distinct right-hand sides multiply these costs. These are cost descriptions, not measured RSS or universal runtime rankings.

5.3 Real-Leja interpolation

The real-Leja backend interpolates the coefficient function $C(z)$ of an action directly on the scaled spectral interval $[ha, hb]$ by Newton interpolation at a real Leja sequence [5], with the spectral bounds supplied by the user or estimated by a Lanczos eigenvalue solver, a stopping rule on the Newton increments, and a maximum degree of 80 by default. There is no substepping in this path. When the maximum degree is reached before the stopping rule is met, the backend raises a convergence error, which the adaptive solvers treat as a rejected step. With tolerance 10^{-10} the default degree covers scaled spectral widths of a few hundred: degree 69 is used at width 240 on a dissipative operator and the cap is reached near width 300, while the canonical problem of Section 7.4 reaches width 53 and the broadband problem of Section 7.6 width 210. The option `target_width` belongs to an experimental augmented-exponential path with substepping that the public coefficient action does not use, so the target width recorded in the canonical campaign had no effect on its computations. The practical attraction of the realization is different from Krylov: with reliable real spectral bounds, repeated sparse products and vector recurrences avoid the growing orthogonalization cost of a Krylov basis, at the price of a degree that grows with the spectral width.

Leja-based exponential integration is also the basis of LeXInt [7]; PyEXPINT does not import or copy LeXInt source.

The manuscript does not state that Leja is intrinsically faster than Krylov, or vice versa. The canonical timing experiment favours the KIOPS-type backend in all six tested configurations, the broadband experiment of Section 7.6 shows the ordering reversing with the tolerance, and the development records of the supplement contain further crossovers with problem size and spectral width.

5.4 SciPy hybrid and pure exponential actions

The SciPy/KIOPS hybrid delegates suitable pure exponential actions to SciPy's `expm_multiply` path and uses the KIOPS-type machinery for the remaining φ combinations. The SciPy exponential-action algorithm is itself based on norm estimation and scaling ideas that avoid forming e^A [1]. This backend is reported with a counting caveat: the PyEXPINT counter does not include the internal products performed by SciPy. For that reason hybrid counts are not used as a total-work ranking in the canonical timing discussion.

5.5 Automatic and calibrated dispatch

`AutoBackend` is a dispatch policy, not another matrix-function algorithm. Its rule-based policy selects a diagonal realization for a one-dimensional diagonal array, a dense realization for sufficiently small dense matrices, real Leja for sufficiently large Hermitian sparse problems with a moderate scaled spectral width, and the KIOPS-type backend otherwise. The size and spectral-width thresholds are exposed because the crossover is machine-dependent.

The release also contains a calibrated selector interface that can be fitted to benchmark data for a defined problem class, and the adaptive solvers preserve its calibration record when they clone it. This is intentionally a moderate claim: calibration can encode a useful local decision rule, but the current experiments do not establish universal optimality. The canonical timing campaign in Section 7.4 is a counterweight to any interpretation that the selector must win because it is automatic: there it chooses real Leja in every configuration, and Leja is the slower realization.

5.6 Fusion of compatible actions

Two fusion levels are useful. First, multiple φ terms acting on the same vector can be represented as one `PhiCombination`. Second, the EGLM engine may need a sum

$$\sum_{j=1}^{q} C_j(hL)v_j, \tag{8}$$

with distinct right-hand sides. The fused operation is defined when all terms share one scale c: with $C_j(z) = \sum_k a_{jk}\varphi_k(cz)$,

$$\sum_{j=1}^{q} C_j(hL)v_j = \sum_{k} \varphi_k(chL)\, b_k, \qquad b_k = \sum_{j=1}^{q} a_{jk}v_j, \tag{9}$$

and a Krylov backend evaluates the right-hand side with one augmented projection of the vectors b_k. When the scales differ, or when a backend offers no fused operation, the engine evaluates the terms independently without changing method semantics. The two paths are algebraically identical, but their approximate realizations need not agree bitwise; the ablation of Section 7.2 reports the agreement actually obtained, below 1.3×10^{-15} in relative terms.

Fusion is therefore an optimization of the common action boundary rather than a special Krogstad or PEC code path. The ablation demonstrates a substantial reduction in recorded products with

both Krylov realizations, while also showing why that reduction should not be relabelled as an equal wall-time speedup.

6 Adaptive integration

PyEXPINT provides two complementary adaptive interfaces. A generic step-doubling controller supplies a reference mechanism applicable to one-step `MethodSpec` objects. A specialized solver implements the embedded ETD34 pair of Whalen, Brio and Moloney [26], built on the scheme of Krogstad [15], and adds backend-aware rejection behaviour. Their structural contracts are compared in Table 4.

Table 4. Structural comparison of the generic step-doubling controller and the embedded ETD34 pair, as implemented in the submitted version. Counts refer to completed trials, not to equal numbers of accepted steps or equal final errors.

Property	Generic step doubling	Embedded ETD34
Applicability	Any one-step MethodSpec	Krogstad fourth-order candidate
Error indicator	$(y_{h/2,h/2} - y_h)/(2^p - 1)$	$h[-\varphi_2(hL) + 4\varphi_3(hL)](N_4 - N_5)$; vanishes for state-independent N
Classical local scaling	$O(h^{p+1})$	$O(h^4)$
Accepted state	Two-half-step solution; no default Richardson extrapolation	Fourth-order Krogstad candidate
Nonlinear evaluations	Twelve per completed Krogstad trial (eleven would suffice)	Four per completed trial after one initial evaluation
Evaluation count	Exact, by wrapping the nonlinear function; failed attempts included	Exact, by wrapping the nonlinear function; failed attempts included
Matrix-action work	$K(h) + 2K(h/2)$	$K(h)$ plus the estimator coefficient action
Endpoint reuse	Not used by this generic implementation	Endpoint FSAL; retain old N_1 on rejection
Action accuracy	Separate solver-level tolerance	Separate solver-level tolerance
Action failure or non-finite trial	Rejected step, retried with half the step	Rejected step, retried with half the step; recorded separately
Termination	Explicit minimum step $\max(h_{\min}, 16\varepsilon T)$; failure there ends the run with a named cause	Same

No guarantee equating requested local tolerance to global solution error is implied.

6.1 Generic step doubling

For a one-step method of order p, the generic controller compares a full step of length h with two steps of length $h/2$. Its error indicator is

$$e_n = \frac{y_{h/2,h/2} - y_h}{2^p - 1}. \tag{10}$$

The implementation accepts the two-half-step state; Richardson extrapolation is not the default. This route is deliberately simple and broad: it provides an adaptive reference for any one-step

catalogue entry without requiring a bespoke embedded formula. Its cost is correspondingly high. A completed Krogstad trial evaluates three complete method steps and twelve nonlinear values; the first stage of the full step and of the first half step coincide, so eleven would suffice, and the generic implementation does not exploit this. The step-doubling results of Section 7 should be read with this unoptimized baseline in mind.

Variable-step history methods are not silently routed through this controller. The published interface excludes variable-step multistep and general linear methods, because changing the step size also changes the representation and transport of history data. Fixed-step history methods remain fully supported.

6.2 Embedded ETD34

The specialized solver advances with the fourth-order Krogstad candidate, whose stiff order is three in the sense of Hochbruck and Ostermann [11], and uses an embedded third-order estimator. With N_4 the nonlinear value at the last internal Krogstad stage and $N_5 = N(t_n + h, y_{n+1})$ evaluated at the fourth-order candidate, the indicator is

$$e_n = h\,[-\varphi_2(hL) + 4\varphi_3(hL)]\,(N_4 - N_5), \tag{11}$$

which is the same expression used by the `rkstiff` implementation of the pair by its first author. A completed trial requires four new nonlinear evaluations after the initial endpoint value. On acceptance, N_5 is reused at the next step; on rejection, the old endpoint value is retained. The scaled error norm uses user-specified relative and absolute tolerances, a fourth-root controller reflects the $O(h^4)$ local scaling of the estimate, and the accepted state remains the fourth-order candidate.

The estimator has a precise limitation. Both nonlinear values in Eq. (11) are evaluated at $t_n + h$. If $N(t, y) = g(t)$ is independent of the state, their difference is zero even when the step has nonzero error. A self-contained example is $L = 0$, $y' = (1 + t)^4$, $y(0) = 0$. Here the Krogstad step reduces to classical RK4, so the first step and the exact solution satisfy

$$\begin{aligned} y_1^{[4]} &= \frac{h}{6}\Big[1 + 4(1 + h/2)^4 + (1 + h)^4\Big], \\ y(h) &= \frac{(1 + h)^5 - 1}{5}, \qquad y_1^{[4]} - y(h) = \frac{h^5}{120}, \qquad e_0 = 0. \end{aligned} \tag{12}$$

This is an algebraic counterexample, not a new time-integration run. It exhibits the estimator degeneracy relevant to the robustness discussion of Zoto and Bowman [27].

For the more general form $N(t, y) = g(t) + F(y)$, only the direct additive value $g(t_n + h)$ cancels. The indicator still contains $F(Y_4) - F(y_{n+1})$. With $d = Y_4 - y_{n+1}$ and F differentiable along the segment between the states,

$$F(Y_4) - F(y_{n+1}) = \left[\int_0^1 DF(y_{n+1} + \theta d)\, d\theta\right] d. \tag{13}$$

The forcing affects the states and their difference and can therefore affect the indicator indirectly. Exact cancellation of the direct forcing term is not a proof that the estimator is blind to all forcing-induced errors when F is nonconstant. Conversely, state dependence alone does not establish robust error detection. The forced comparisons in Section 7.5 and the autonomous example in Section 7.6 are interpreted case by case. Step doubling is the documented alternative when the embedded indicator is unsuitable; replacing the published pair is not a contribution of this article.

6.3 Temporal and matrix-action tolerances

An exponential integrator has two distinct accuracy mechanisms: temporal error control and numerical approximation of matrix-function actions. ETD34 therefore creates a work backend

whose action tolerance is

$$\tau_{\mathrm{act}} = \min(\tau_{\max}, \max(\tau_{\min}, f_{\mathrm{act}}\,\mathrm{rtol})), \tag{14}$$

where f_{act} is an explicit action-tolerance fraction. The package default is 0.05; the canonical timing campaign deliberately uses 0.01 and records that value as part of the experiment configuration. Neither number is claimed to be universally optimal. Supplementary Figure S3 retains the tolerance-coupling sweep that motivated this separation. The work backend is obtained from the user's backend through a single hook, `with_tolerance`, so that external backends participate in the adaptive solvers on the same footing as the built-in ones, and a calibrated selector keeps its calibration record when cloned.

6.4 Failed actions, non-finite trials and termination

Iterative matrix-function algorithms can fail to meet their stopping criterion for a particular trial step. In ETD34, convergence exceptions from Leja, KIOPS-type or Krylov actions, and trials that produce non-finite values, are treated as rejected time steps: the solver halves the step and retries rather than terminating the integration. Both kinds of rejection are recorded separately from ordinary error-estimator rejections, and the nonlinear evaluations and matrix-vector products spent on failed attempts are counted, not estimated.

Termination is explicit. Both adaptive solvers use the minimum step $\max(h_{\min}, 16\varepsilon T)$ with $T = \max(|t_0|, |t_f|, t_f - t_0)$, and an attempt that fails or is rejected at that minimum ends the integration with an error that names the cause. A maximum step below the smallest resolvable step of the interval is rejected before integrating, a step that would not advance the floating-point time is reported, and a failure that leaves a step too small to reach t_f within the attempt budget stops the integration immediately instead of exhausting the budget. All option values are validated, including finiteness. These rules replace the behaviour of the first release, in which a persistent failure at the minimum step consumed the whole attempt budget.

6.5 Cost-aware control is experimental

The implementation also contains a cost-aware proposal based on observed work per unit simulated time. It can constrain the accuracy-based step proposal when a local log–log cost gradient suggests a cheaper step size. The work of an attempt is the increment of the backend's own matrix-vector counter, and the controller refuses to run in matvec units with a backend that reports no counter; it never substitutes wall-clock time for a missing count. The stress test of Section 7 shows why this feature is not a headline performance claim: the matvec proxy decreases while recorded wall time can increase, because the controller takes more steps and nonlinear evaluations. We therefore retain it as an instrumented diagnostic, not as evidence of lower solve time.

7 Numerical experiments

The experiments distinguish three questions: whether the recorded method is implemented consistently, how algebraic execution choices change its work, and what a specified execution environment reports about elapsed time. These questions require different controls. Classical and stiff-order experiments use analytic or directly diagonalized references. The fusion experiment changes only one execution option. The method comparison fixes the action backend and the numerical integration window. The canonical timing experiment retains separate campaigns and paired measurement blocks instead of pooling all observations into a single ranking.

The supplied numerical records have heterogeneous provenance. Table 5 distinguishes historical data, author-reported candidate runs and postprocessing. Classical and stiff-order regeneration, the new fusion and generalized Lawson runs, the numerical-startup study and a differential check of 24

Table 5. Experimental protocols and provenance. Candidate reproduction, recorded historical execution and data-to-figure regeneration are distinct. The private companion supplies the sources and validation records referred to below; its 24-case differential check is not a rerun of the 108-case fixed-step campaign.

Block	Protocol and manuscript record	Execution status
Classical order	Scalar Riccati; four steps; `classical_selected.json`.	Candidate comparison: 16 errors, log under `evidence/cheap_*`. Fig. 2.
Stiff order	Two problems; 200 points, DST-I; `stiff_order.json`.	Candidate comparison of 72 errors in the same recorded run. Fig. 2.
Fusion	Two Krylov backends; $n = 256$, $T = 0.3$, $h = 0.05$.	Original record and candidate rerun retained; totals and state comparisons, not every action's dimension. Fig. 3.
Method work–precision	$n = 511$, 27 methods, four steps, exact history.	Historical 108-case record, reanalysed; no separate candidate-rerun log is supplied. Fig. 4.
Canonical timing	Three campaigns; six cases, four backends; 72 raw files.	Historical elapsed times. Candidate error/count comparison has 72 replica checks, not new timings. Fig. 5.
Forced adaptivity	Periodic/stiff manufactured problems; two controllers.	Historical records, reanalysed, not relabelled as candidate runs. Table 6.
Broadband	Allen–Cahn, $n = 1024$, six tolerances, 18 configurations.	Original workstation JSON, driver, initial/reference states and five times per configuration. Fig. 6.
Startup	Scalar problem on $[0, 1]$; exact/numerical history.	Supplied error arrays and candidate log under `evidence/c2c4_*`. Table 7.
Generalized Lawson	Two six-dimensional problems, four steps.	Supplied data and candidate run under `evidence/cheap_*`. Supplementary Table S2.

timing configurations are reported in the candidate description. Their execution logs and candidate snapshot are now supplied in the private companion for software-level verification. The 108-case fixed-step campaign, canonical timings and forced adaptive comparisons retain their historical identity; a check of the 24 timing configurations is not a check of the 108 fixed-step cases. All plots in this package are generated from the supplied records. Regeneration of a plot establishes the data-to-plot transformation, not reexecution of the underlying integrator. The original Allen–Cahn record is supplied with the driver and configuration described in Section 7.6.

7.1 Classical convergence and stiff-order diagnostics

The classical scalar problem is

$$y' = -y + y^2, \qquad y(0) = 0.4, \qquad y(t) = \frac{1}{1 + 1.5e^t}, \qquad 0 \le t \le 1.6. \tag{15}$$

The four step sizes are $0.2, 0.1, 0.05, 0.025$. The top panel of Figure 2 selects ETD4RK, Krogstad, HochOst4 and ETD5RKF. The first three exhibit the expected fourth-order refinement, while ETD5RKF exhibits fifth-order refinement on this example. The separate final-state comparison in Section 4.3 does not supply the complete EXPINT refinement sequence used to compare convergence

orders. This comparison also illustrates why an order label alone does not rank errors at a specified step: the fifth-order method need not have the smallest error in the displayed range. The full catalogue metadata and a generic convergence test of the generalized Lawson families on a six-dimensional system are retained in Supplementary Tables S1 and S2.

For the stiff tests, the manufactured solution is $u_*(x,t) = x(1-x)e^t$ on $(0,1)$ with homogeneous Dirichlet boundary values, as in Hochbruck and Ostermann [11]. The two nonlinear terms are, respectively, $1/(1+u^2)$ and the spatial integral of u, together with the manufactured source. The source is formed at the semidiscrete level. A centred second-difference Laplacian on 200 interior points is diagonalized by the discrete sine transform of type I, and its φ-actions are evaluated with DiagonalBackend. Thus an iterative Krylov or Leja stopping tolerance is not part of these temporal refinement curves. The steps range from $1/20$ to $1/640$ by successive halving.

The middle and bottom panels of Figure 2 show that the classical order does not by itself predict the observed stiff behaviour. In the nonlocal test the last adjacent slope of ETD4RK is about 2.50; the corresponding Krogstad slope is about 3.36, whereas HochOst4 retains approximately fourth-order behaviour. These numbers are finite-grid, finite-step diagnostics, not new order theorems. Some of the finest errors are already of order 10^{-13}, so their last differences should not be used to upgrade a method classification. RKMK2e and RKMK4t show approximately second-order decay in these tests without thereby satisfying the strong second-order moment condition away from $z = 0$. Their recorded strong and weak distinction is retained in Table 2. All plotted points are preserved; the reference slopes drawn in the panels are guides, not fits.

7.2 Ablation of algebraic action fusion

The fusion experiment compares one realization with compatible action sums evaluated independently or jointly. It uses the unit tridiagonal operator $\mathrm{tridiag}(1,-2,1)$ of dimension 256, the reaction $N(u) = 0.08u^2$, the initial state $u_0 = 0.05 + 0.01\sin s$ with s uniform on $[0,\pi]$, final time $T = 0.3$, step $h = 0.05$ and action tolerance 10^{-10}. The seven retained methods include one-step and general linear specifications; their definitions and the time grid are unchanged by the fusion switch. Full Arnoldi is switched with its own fusion flag; the KIOPS-type backend has no switch, and its unfused path is obtained by hiding the fused operation from the bound context.

Figure 3 reports the counts. With full Arnoldi, Krogstad decreases from 756 to 486 products, PEC423 from 774 to 510 and EGLM433 from 990 to 609; across the seven methods the reductions range from 13.4% to 38.5%. With the KIOPS-type backend the reductions range from 11.8% to 44.2%. In every case the fused and unfused final states agree to a relative difference below 1.3×10^{-15}. The selection contains three methods without history, ETD4RK, Krogstad and HochOst4, and four with nonlinear history, PEC423, EGLM433, GenLawson43 and ModGenLawson43. The archived record supplies aggregate counts and final-state differences, not projected dimensions for every action or trials with different initial states. Equal totals therefore do not establish vector-independent Krylov dimensions or isolate the cause of a difference from an earlier run.

The outcome is deliberately stated in terms of the measured work component. A smaller count does not imply the same percentage reduction in runtime: forming and evaluating a larger augmented projected problem has its own cost, and the development records contain cases in which fusion did not reduce elapsed time. The portable conclusion is that the generic action boundary can exploit a semantics-preserving transformation whose computational benefit must be assessed with more than one observable.

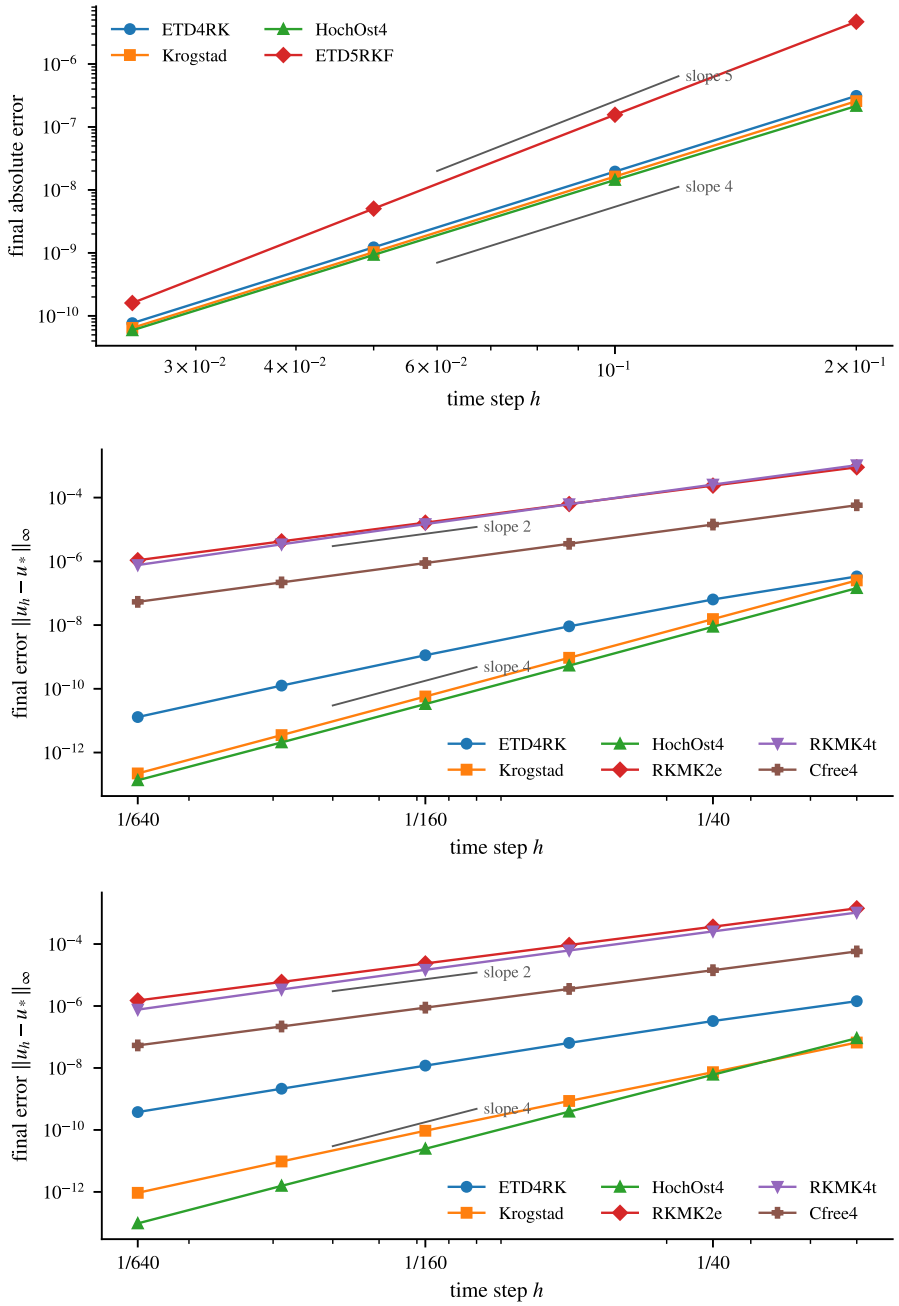


Fig. 2. Classical and stiff temporal refinement. Top: classical scalar test, four one-step methods. Middle and bottom: local and nonlocal manufactured problems of Hochbruck–Ostermann type with 200 interior points and exact diagonal actions. Each marker is a final error; connecting segments identify a refinement sequence and the short straight lines are reference slopes. Finest errors approaching 10^{-13} are retained as data.

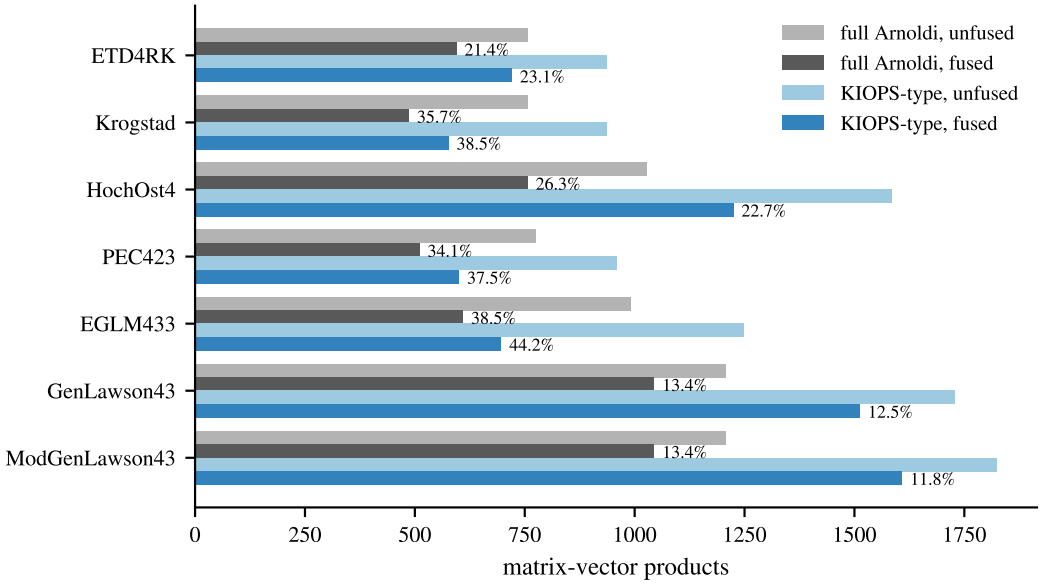


Fig. 3. Fusion ablation. Bars are matrix-vector counts without and with fusion for full Arnoldi and for the KIOPS-type backend; annotations give the percentage reduction. The bars are not wall-clock speedups.

7.3 Method work–precision with a fixed action backend

The method comparison uses 511 interior points and the nonseparable manufactured solution

$$u_*(x,t) = e^t q_1(x) + \tfrac{1}{4} e^{2t} q_2(x), \qquad q_1 = x(1-x), \qquad q_2 = x^2(1-x)^2. \tag{16}$$

For the discrete Laplacian L, the forcing is computed as $u_*'(t) - Lu_*(t) - 1/(1+u_*(t)^2)$, rather than by substituting the continuous second derivative. The KIOPS-type action tolerance is fixed at 10^{-13}. All methods use exact history and the same numerical integration window $[0.5, 1]$. Startup history lies before this window and is not presented as the cost of a practical numerical startup; Section 7.7 shows what the default numerical startup does to the high-order entries. The four steps are $0.1, 0.05, 0.025, 0.0125$. The campaign reports a semidiscrete residual of 1.22×10^{-15} and no nonconverged matrix-function action in any of its 108 method–step cases.

Figure 4 selects ETD4RK, Krogstad, HochOst4, ABNorsett4, PEC625, PEC726, PECEC635 and PECEC736 from the 27 methods of the campaign. Every displayed curve contains the four step sizes. Open enlarged markers identify points at or below the precision cutoff 5×10^{-12}, which is tied to the action tolerance; these points remain visible but are excluded from the non-dominated curves. The cutoff is a diagnostic for this data set, not a solver tolerance or accuracy guarantee.

The result depends materially on the chosen work observable. In the error–matvec plane, the non-dominated subset of the eight displayed methods starts with the four ABNorsett4 refinements and then contains PEC625 at $h = 0.05$, PEC726 at $h = 0.05$, PEC625 at $h = 0.025$ and PECEC635 at $h = 0.025$. In the error–nonlinear-evaluation plane, the corresponding sequence contains ABNorsett4 at $h = 0.1$, PEC726 and PECEC736 at $h = 0.1$, PEC726 and PECEC736 at $h = 0.05$, and PEC625 and PECEC635 at $h = 0.025$. Thus a method that is efficient in one recorded component need not be efficient in another. The figure is a work decomposition under one fixed matrix-action realization, not a universal ranking of the time integrators. Two further observations from the same campaign are instructive. ETD5RKF, of classical order five, converges with adjacent slopes 4.66,

3.27 and 2.97 on this stiff problem, the clearest illustration in the catalogue of why classical and stiff orders are recorded separately. The generalized Lawson entries GenLawson44 and GenLawson45 show slopes rising towards 5 and 6 here, one above their recorded orders, whereas on the generic six-dimensional test of Supplementary Table S2 their slopes decrease towards the recorded 4 and 5; the recorded orders are kept, and the parabolic behaviour is reported as a problem-dependent observation.

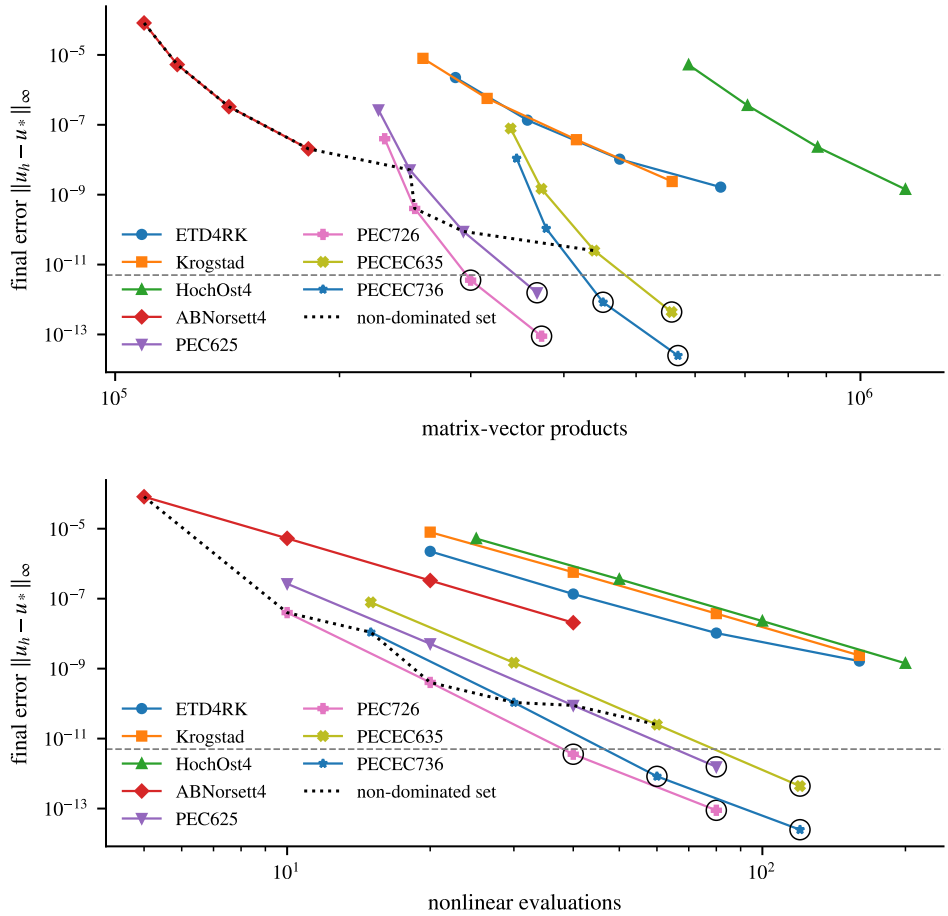


Fig. 4. Method work–precision on the nonseparable semidiscrete problem with exact history and a fixed KIOPS-type realization. Top: final error versus matrix-vector products. Bottom: final error versus nonlinear evaluations. Eight methods and four step sizes are shown. The dotted curve is the non-dominated set for the displayed methods and the corresponding work observable. Enlarged open markers lie at or below the precision cutoff and are not used to extend that curve.

7.4 Replicated paired timing

The canonical timing problem is a periodic two-dimensional manufactured reaction–diffusion system on $[0, 2\pi)^2$, with $\nu = 0.02$, $T = 0.5$ and

$$u_*(x, y, t) = 0.2e^{-t} \sin(x) \sin(y). \tag{17}$$

The spatial operator is the Kronecker-sum second-difference Laplacian; the nonlinear part is u^2 plus the semidiscrete source required by u_*. The measured operation is the ETD34 solve only: problem construction and explicit garbage collection are outside the timer. Each configuration has one warmup and 12 timed blocks, and the backend order is balanced cyclically.

The grids have 64^2, 128^2 and 256^2 unknowns, and the requested tolerances are 10^{-5} and 10^{-7}. The four choices are KIOPS-type, real Leja, the SciPy/KIOPS hybrid and the rule-based selector. The absolute tolerance is 10^{-3} times the relative one; the action-tolerance fraction is 0.01, with the documented clipping rule; the initial step is 0.1 and the maximum step is 0.2. Explicit Leja and the Leja realization chosen by the selector use a maximum degree of 80 without substepping, which covers the scaled spectral widths of this problem, at most 53 with $h = 0.2$ on the finest grid. These are experiment settings, not inferred defaults.

The campaigns ran on the Picasso cluster of the University of Málaga, as job 2203999 on node sd041 and jobs 2206865 and 2207024 on node sd004, each on one allocated core of an Intel Xeon Gold 6230R at 2.10 GHz with thread counts set to one, under Python 3.11.4, NumPy 1.24.3 and SciPy 1.10.1. They are three separate campaigns, not three independent machine types, and no exclusive node occupancy is asserted. For backend B, replica r, case c and paired block b, the timing observable is

$$R_{r,c,b}(B) = t_{r,c,b}(B)/t_{r,c,b}(\mathrm{KIOPS}). \tag{18}$$

Figure 5 preserves the three per-replica medians and the within-replica first and third quartiles for every case. The 54 medians all exceed one, between 1.044 and 1.484, so the KIOPS-type backend was faster than each alternative in these configurations and on this machine.

Three facts qualify that statement. First, the comparison is at closely comparable accuracy: the final errors of the four realizations differ from the KIOPS-type error by ratios between 0.960 and 1.0005 in every configuration, with the same numbers of nonlinear evaluations, 13 at 10^{-5} and 29 at 10^{-7}, and of accepted and rejected steps. The re-run with the submitted version shows that the step sequences are identical at 10^{-5}, namely 0.1, 0.2, 0.2, and agree to five decimal places at 10^{-7}. Second, the work counters explain the ordering. Real Leja performs 1.63 to 3.11 times as many matrix-vector products as the KIOPS-type backend but takes only 1.08 to 1.47 times as long, so the amortized solve time per recorded product is 0.45 to 0.81 times that of the Krylov realization; this amortized ratio contains all the work of a solve, not the cost of one product. The KIOPS-type counts depend weakly on the grid, 542, 542 and 558 products at 10^{-7}, whereas the Leja counts grow with it, 884, 1109 and 1720. This is consistent with the nature of the manufactured solution, an eigenvector of the periodic second-difference Laplacian whose square adds only the Fourier modes of wavenumber 0 and 2, so that the vectors acted upon are dominated by a few modes, a situation in which a Krylov space of small dimension captures the action while a polynomial interpolant on the whole spectral interval must still resolve the width of the interval. The campaign therefore does not inform about operators with broadband data, which is why Section 7.6 adds one. Third, at the looser tolerance the controller is maximum-step-limited: thirteen evaluations are three attempts, and with the initial step 0.1 and the maximum step 0.2 the only possible sequence is 0.1, 0.2, 0.2, so half of the campaign exercises the error control only through the acceptance of those steps.

The selector chose real Leja in all 18 configurations and its medians exceed those of explicit Leja by 0.3% to 3.7% in ratios of unpaired medians; the bottom panel of Figure 5 therefore measures Leja plus whatever the dispatch adds, without a paired isolation of that overhead. The rule-based thresholds are a counterweight to any reading in which automatic selection must win: on this problem the rule picks the slower backend, and the calibrated selector of Section 5 is not evaluated here. The replicas are informative about variability. Within a replica the interquartile ranges of the paired ratios are 0.13% to 2.63% of the median, but between replicas the medians differ by 2.6% to

5.6%, with the sd041 replica lowest in 14 of 18 cases, while the absolute medians differ by 17% to 30% between the two nodes. Three campaigns on two nodes do not support a causal attribution of these differences, but the between-replica spread is a better indication of the variability of a paired ratio than the within-replica quartiles, and the conclusion survives it because the smallest median is 1.044. The complete 72 configuration summaries appear in Supplementary Table S3 with errors and work counters beside time. Hybrid matvecs are not used as a total-work competitor because the counter omits the internal SciPy work. Development crossovers measured under other protocols remain in Supplementary Figure S1.

7.5 Adaptive work and achieved accuracy

The structural comparison in Table 4 is complemented by the four periodic pairs in Table 6. The record contains two requested tolerances per backend and controller. It does not support a smooth work–precision curve or a fixed percentage advantage at matched final error. At rtol 10^{-4} the KIOPS-type runs use 731 and 334 matvecs, but their final errors are approximately 2.01×10^{-7} and 2.30×10^{-6}.

Table 6. Periodic manufactured problem ($n = 256$): step doubling (SD) versus ETD34 with accuracy control. Arrows indicate SD → ETD34. Comparisons use the same requested tolerance but explicitly retain the different achieved errors. Counts are registered quantities; the historical file does not contain a separate backend-failure count.

Backend	rtol	Error SD	Error ETD34	Matvecs	N calls
KIOPS-type	1×10^{-4}	2.01×10^{-7}	2.30×10^{-6}	731 → 334	36 → 13
KIOPS-type	1×10^{-6}	3.99×10^{-8}	5.06×10^{-8}	1191 → 718	48 → 33
Leja	1×10^{-4}	2.63×10^{-5}	1.14×10^{-5}	3249 → 1375	36 → 13
Leja	1×10^{-6}	4.60×10^{-8}	5.71×10^{-8}	5387 → 3447	48 → 33

These comparisons are also exposed to the limitation that Section 6 makes explicit. The problems are manufactured, so their source terms are part of N. The direct additive source cancels from the same-time nonlinear difference used by ETD34, but its influence through the stage states does not generally cancel, as Eq. (13) shows. Step doubling compares two complete step constructions instead. The records do not isolate the effect: ETD34 reaches larger final errors than step doubling in the KIOPS-type periodic cases, but smaller ones in the Leja case at 10^{-4} and in the nonlocal stiff test, so an underestimate of the error is a possible contribution to its lower work, not an established cause. The broadband problem of Section 7.6, where N depends on the state only, shows the pair without an additive manufactured source, but does not by itself prove a robustness property for every autonomous nonlinearity. The stiff adaptive tests supply a further qualification. In the nonlocal problem at rtol 10^{-6}, ETD34 records 97 nonlinear evaluations versus 84 for step doubling, while attaining errors 3.51×10^{-8} and 2.67×10^{-7}. Thus the lower work per completed trial is not a theorem about total solve work. Supplementary Table S4 retains both local and nonlocal tests, including this less favourable count comparison.

The cost-aware stress test reinforces the same distinction. Its matvec proxy decreases by approximately 8.0% and 21.4%, yet the recorded wall times increase by approximately 9.7% and 3.8%. This is not promoted to a replicated performance result. It demonstrates why one counter is insufficient for a cost model that also contains nonlinear evaluations, projected work, vector operations and implementation overhead.

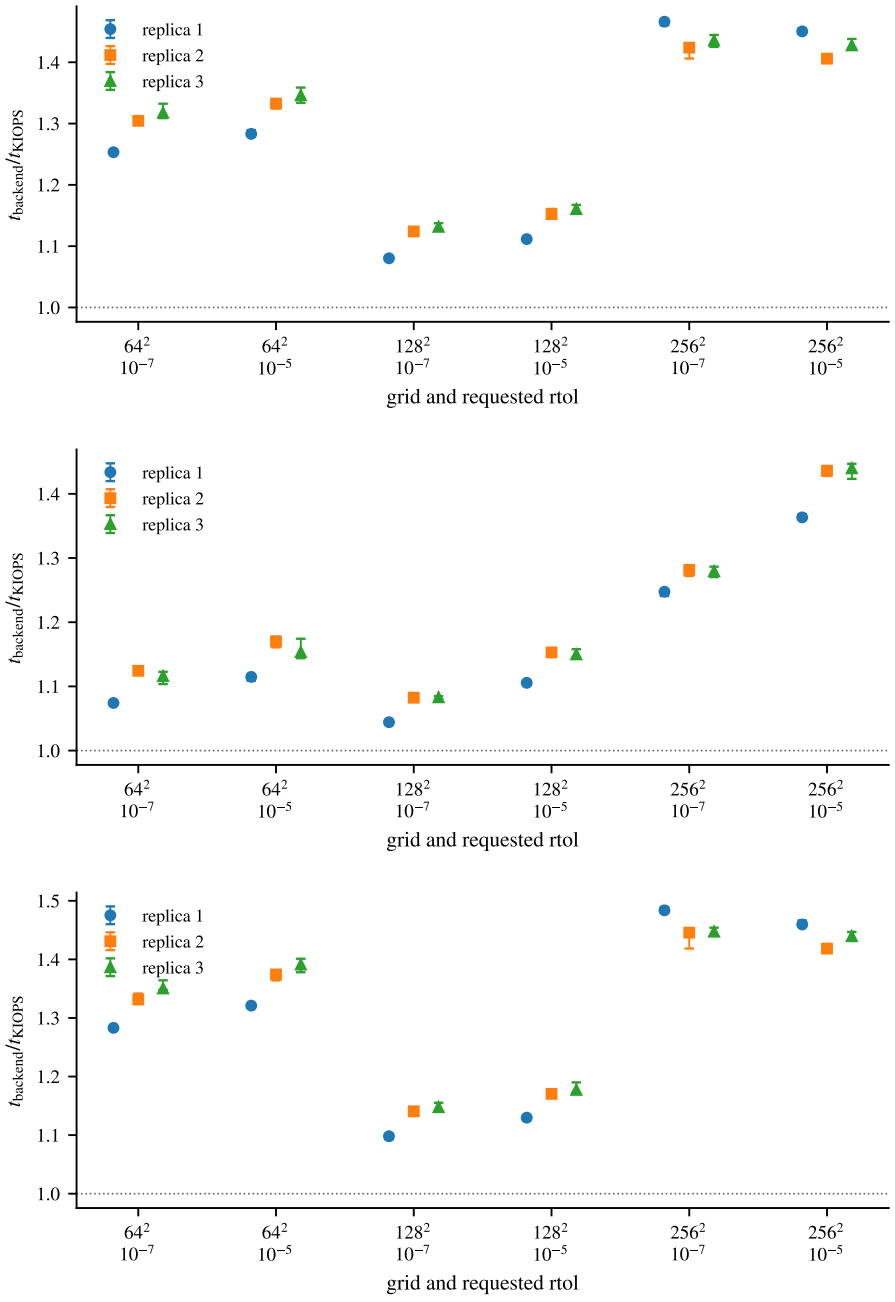


Fig. 5. Canonical paired timing summaries. Top: real Leja. Middle: SciPy/KIOPS hybrid. Bottom: rule-based selector, which chose Leja in every case. Every marker is one replica's median of 12 within-case paired ratios; bars span its first to third quartile. Ratios above one mean a longer solve time than the KIOPS-type backend. Within-replica quartiles are not confidence bounds; the between-replica spread is reported in the text.

7.6 A broadband problem without manufactured forcing

The Allen–Cahn equation $u_t = \varepsilon u_{xx} + u - u^3$ on the periodic interval $[0, 1)$ is discretized by second differences on 1024 points, with $\varepsilon = 2.5 \times 10^{-4}$ and $T = 1$. The initial state is $u_{0,j} = \tanh(4f_j/\max_k |f_k|)$, where f is a sum of six sine and cosine modes. The original record specifies seed 7, all twelve coefficients, the construction rule and the full initial vector. The driver is `software/benchmarks/broadband_allen_cahn.py` in the private companion. The initial and reference-final vectors have six sign changes and require 27 and 18 positive-frequency bins, respectively, to accumulate 99.99% of the recorded nonconstant spectral energy. Thus this experiment is not confined to the single Fourier mode of the canonical timing problem.

The reference record stores the 1024-component result of fixed-step HochOst4 with 4096 steps and KIOPS-type action tolerance 10^{-13}. It reports an infinity-norm difference of 3.897×10^{-14} from the 2048-step comparison and final maximum amplitude 0.9995349106. The finer reference state is supplied; the coarser reference state is not, so the comparison difference is a retained diagnostic, not independently recomputed from the two states in this package. Similarly, per-solve final errors are outputs of the driver; the record does not retain all 18 final solution vectors.

ETD34 with KIOPS-type and real-Leja actions is compared with Krogstad step doubling with KIOPS-type actions, at six relative tolerances from 10^{-4} to 10^{-9}. The settings are atol = 10^{-3}rtol, $h_0 = 0.01$, $h_{\max} = 0.2$ and action fraction 0.01. The original JSON contains the options, achieved errors, counts, accepted step sequences and five individual wall times for all 18 configurations. The five times are measurements of each complete solve, not paired observations across configurations. The recorded execution used Python 3.11.16, NumPy 1.24.3 and SciPy 1.10.1 with single-threaded BLAS on an Intel Core Ultra 9 185H workstation. The recorded source commit is `f6ef3de1a85005d93f8a3e3744514dce65b9f68a`; it precedes the final review snapshot, and no new timing of that snapshot is claimed.

Figure 6 uses the full original record rather than its previous provisional transcription. At 10^{-4}, Leja takes 0.631 times the KIOPS-type time but reaches an error 4.723 times larger: that point is not a matched-accuracy speed comparison. At 10^{-5}, the error and time ratios are 0.973 and 0.655, while at 10^{-9} they are 0.997 and 2.081. The latter pairs show a change of the faster realization at closely comparable achieved accuracy within this experiment. At 10^{-6} the counts are close but not equal, 4891 for Leja and 4954 for KIOPS-type. Counts, errors and times are distinct observables; the curves do not imply monotonic changes at every refinement or a universal backend ordering.

7.7 End-to-end order with numerical startup

The high-order history entries of Section 7.3 were run with exact history. Table 7 reports what the default numerical startup does to them on the classical scalar problem of Section 7.1, integrated to $t = 1$ with exact diagonal actions and steps from 0.1 to 0.0125. With one HochOst4 step per startup interval, the selected entries of order six and seven have slopes between 4.55 and 4.80 with numerical startup, compared with 5.86 to 6.84 for exact history, on the intermediate pair $h = 0.05$ to $h = 0.025$. The table also shows both errors at $h = 0.0125$. For PECEC635 these are 8.33×10^{-13} with numerical startup and 9.44×10^{-16} with exact history. Their ratio is arithmetically about 882, but the denominator is close to machine precision; it is not a stable quantitative factor of startup degradation. Reporting the errors separately avoids that interpretation. The three displayed fourth-order controls have very similar intermediate slopes, while the fifth-order cases show method-dependent error constants. These slopes are finite observations consistent with the bound $\min(p, q + 1)$ of Section 3; they do not prove the asymptotic order, and the bound itself allows a higher observed order when the leading startup term vanishes for a particular problem. They do

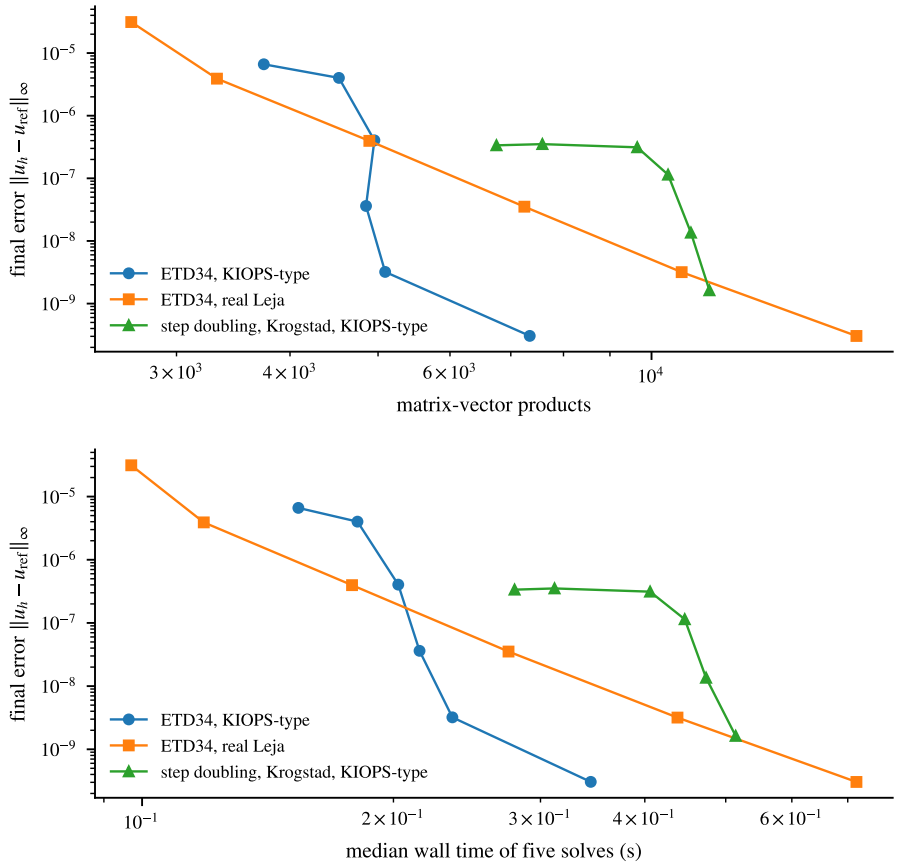


Fig. 6. Broadband Allen–Cahn comparison from the original workstation record. Top: final error versus recorded matrix-vector products. Bottom: final error versus the median of five complete-solve wall times; all 90 individual times are archived. Segments connect the six requested tolerances without asserting monotonicity. Ratios at equal requested tolerance are not automatically comparisons at equal achieved accuracy.

show that the default startup does not in general deliver the nominal orders six and seven, and that more accurate starting values are needed for that.

7.8 Reproducibility and limits of interpretation

The supplied figure generator rebuilds the plotted numerical panels. The table generator rebuilds four specified tables, and a separate parser builds Supplementary Table S6 from the cross-check summary. Other tables are supplied as LaTeX with their associated records where available. These postprocessing operations are distinct from software execution. The supplied private snapshot and validation logs provide the latter evidence within the environments and configurations identified in Section 9. The evidence supports a common validated execution framework, a measurable action-fusion transformation and reproducible, environment-specific work and timing diagnostics. It does not support superiority over external packages, distributed-memory scalability, certified action-error bounds for every stopping criterion, globally optimal automatic selection or a universal ETD34 efficiency factor.

Table 7. End-to-end convergence on the scalar problem, $0 \le t \le 1$, with exact history (E) and one HochOst4 step per numerical-startup interval (S). Slopes use the intermediate pair $h = 0.05$ to $h = 0.025$; the last two columns give both errors at $h = 0.0125$. Some exact-history errors are close to machine precision, so their ratios are not interpreted as stable factors of degradation. Generated from the complete error arrays in `c4_startup_order.json`; the named simulation driver and its execution log belong to the separate software companion.

Method	p	Slope E	Slope S	Error E	Error S
ABNorsett4	4	3.92	3.92	6.47e-10	6.47e-10
EGLM433	4	3.93	3.97	2.98e-11	3.03e-11
PEC423	4	3.92	3.95	4.62e-11	4.66e-11
GenLawson45	5	4.96	4.82	6.40e-14	8.96e-13
ModGenLawson44	5	4.89	4.86	9.00e-14	7.23e-13
PEC524	5	4.87	4.87	3.13e-12	3.77e-12
PECEC534	5	4.94	4.92	1.30e-12	1.93e-12
ModGenLawson45	6	5.88	4.80	1.42e-15	8.33e-13
PEC625	6	5.86	4.55	5.15e-14	7.81e-13
PECEC635	6	6.25	4.79	9.44e-16	8.33e-13
PEC726	7	6.68	4.77	3.22e-15	1.03e-12
PECEC736	7	6.84	4.76	3.39e-15	1.03e-12

Table 8. Functional comparison with related exponential-integrator software, from the cited documentation and sources; no runtime comparison is implied.

Package	Language	Integrators	Matrix-function actions	Step control and instrumentation
EXPINT [3]	MATLAB	47 exponential RK, Lawson, Adams, predictor–corrector, general linear and Lie-group schemes in one GLM format	Padé approximants and dense or diagonal exponentials, computed per step	Fixed step; no work counters
EXPODE [14]	MATLAB	Several classes of exponential integrators	Direct and Krylov evaluation	Adaptivity for some methods
LeXInt [7, 8]	Python, C++/CUDA	Exponential Rosenbrock and EPIRK schemes	Leja interpolation	Adaptive variants; GPU version
rkstiff [25]	Python	ETD and integrating-factor Runge–Kutta pairs ETD34, ETD35, IF34, IF45DP	Diagonal or dense matrix operators	Embedded adaptive pairs
OrdinaryDiffEq.jl [20]	Julia	Lawson, Nørsett, ETDRK2–4, HochOst4, Exp4, EPIRK and adaptive Exprb schemes	Cached dense functions or Krylov actions switched by one option	Adaptive for the Exprb pairs; ecosystem instrumentation
PyEXPINT	Python	The 47-entry EXPINT catalogue as one explicit EGLM form, with classical, stiff and weak order metadata	Dense, diagonal, full Arnoldi, KIOPS-type, real Leja and hybrid realizations behind one action contract, with algebraic fusion and an extension hook	Fixed step with explicit startup, step doubling, ETD34 with action-tolerance coupling and failure rejection; authoritative work counters and provenance

8 Relationship to existing exponential-integrator software

Table 8 summarizes the functional position of PyEXPINT with respect to the packages discussed below. The comparison is deliberately not a runtime ranking.

The most direct historical reference is EXPINT itself [3]. EXPINT was designed to make a broad set of exponential Runge–Kutta, multistep and general linear formulas easy to deploy and test in MATLAB, with the matrix-function technology of its period, Padé approximants of the φ functions [23]. PyEXPINT preserves the value of the broad catalogue but changes the software decomposition: method coefficients become backend-independent analytic expressions, all explicit methods pass through one external-state engine, and large-scale matrix-function-vector algorithms are interchangeable realizations rather than embedded method code. The relationship is established by execution: Section 4.3 runs the original package against the reconstruction, method by method.

EXPODE pursued another broad-toolbox design in MATLAB and supports several classes of exponential integrators together with direct and Krylov-based matrix-function evaluation, including

adaptivity for some methods [14]. The overlap is conceptual: both projects treat experimentation across integrator classes as a software goal. PyEXPINT differs in the unification around an explicit EGLM registry, the reconstructed and cross-checked EXPINT catalogue, its action algebra, and the work and provenance instrumentation.

At the matrix-function level, Expokit [22], Algorithm 919 and KIOPS are not competitors to the PyEXPINT architecture so much as exemplars of why the action boundary is needed. Algorithm 919 develops an adaptive Krylov procedure for combinations of the exponential and φ functions [17]; KIOPS extends the augmented-Krylov strategy with adaptive incomplete orthogonalization and substepping for exponential-integrator workloads [9]. The PyEXPINT KIOPS-type backend is an independent implementation derived from the published algorithmic ideas, which is why it is not called KIOPS, and it is not claimed to outperform the reference implementation. The real-Leja backend follows the interpolation of Caliari, Vianello and Bergamaschi [5] with the backward-error view of Caliari, Kandolf, Ostermann and Rainer [4]; LeXInt provides Leja-based exponential Rosenbrock and EPIRK integrators in Python and a later C++/CUDA version [7, 8], and shows that polynomial actions can be competitive with Krylov methods. In PyEXPINT Leja is one realization available to the same historical method definitions rather than the defining representation of the library.

The closest Python neighbour of the adaptive solver is `rkstiff`, which implements the ETD34 pair of Whalen, Brio and Moloney [26] together with ETD35, IF34 and IF45DP for diagonal or dense matrix linear operators [25]. PyEXPINT implements the same ETD34 estimator for general operators and documents its blindness to state-independent forcing, which the robustness analysis of Zoto and Bowman [27] addresses with new pairs.

SciPy provides a widely used exponential-action implementation through `expm_multiply` [1, 24]. PyEXPINT uses that functionality in its hybrid realization instead of replacing the scientific Python ecosystem's mature pure-exponential path. NumPy and SciPy also provide the array, dense and sparse, transform and linear-algebra substrate for the package [10, 24].

The Julia ecosystem separates reusable matrix-function actions from time-integration packages in a way that is closest to the design adopted here. ExponentialUtilities.jl supplies matrix exponential and φ-action utilities, including KIOPS, for the SciML integrators [21], and OrdinaryDiffEq.jl offers Lawson, Nørsett, ETDRK, HochOst4, Exp4, EPIRK and adaptive Exprb schemes whose actions are either cached dense functions or Krylov evaluations selected by one option [20]. That switch is the same separation of method and realization pursued here with six realizations, an explicit algebra of combinations, fusion and an extension hook. The niche of PyEXPINT is the combination of a validated historical EGLM registry, one execution engine, multiple action realizations, startup and history semantics, and benchmark instrumentation in a Python package.

9 Reproducibility, availability and limitations

9.1 Identified private software snapshot

The accompanying archive `pyexpint_review_package_20261011.zip` contains the software, original experiment records, distribution artifacts and validation logs. Its source tree, excluding the packaging-only `SNAPSHOT.txt`, has the Git tree identifier

`9f62cbf6a6455868e358386ccff824507287aeb3`.

This identifier can be reconstructed from the supplied contents and file modes. The recorded preparation commit is `0ad4b3c4786086a04cf22be33375cc9997ff1344`. The manifest covers 317 payload files. Both distributions contain package version metadata 0.9.0, retained from development; this label is not the identity of the corrected snapshot and is not a claim that it is the earlier public release. The 20 Python source files in the supplied wheel match those in `software/src/pyexpint`.

The source distribution likewise matches the corresponding supplied source files apart from generated build metadata. No new public release is implied here. The earlier public 0.9.0 archive, DOI `10.5281/zenodo.22820595` [19], remains a separate historical artifact. Software licensing is MIT and original non-software content is CC BY 4.0; original EXPINT MATLAB sources are not redistributed.

The detailed preparation-environment JUnit records report 275 passing tests from the snapshot and the same 275 identifiers against the installed wheel. The logged environment is Python 3.11.15, NumPy 1.24.3, SciPy 1.10.1 and pytest 7.1.2. Four examples ran outside the repository; the source distribution was also built and installed and ran the no-exact-history example. Scripts print a fixed "WSL" label, but the package explicitly identifies these detailed logs as preparation-container runs. An additional author-workstation summary concerns Python 3.11.16 and earlier commits; it is not substituted for those detailed logs. A Python 3.11–3.14 workflow is configured in the source, but execution of that entire matrix is not established by the supplied candidate records.

9.2 Experiment and manuscript reconstruction

Table 5 distinguishes original records, candidate comparisons and historical results. The full original Allen–Cahn JSON replaces the provisional transcription and records 90 individual times, step sequences, construction coefficients and initial/reference states. Its source identifier and environment remain those of the workstation run. The driver and original records are in the private companion, and the JSON used by Figure 6 is byte-identical to that original. Reference-comparison scalars and per-solve error scalars retain their status as driver outputs when both compared final states are not archived.

The 72 original canonical JSON files are now supplied under `software/reproducibility/toms/rep1/raw` and the corresponding `rep2` and `rep3` directories. They contain 864 complete-solve timing records. Matching block identifiers produces the 54 paired summaries used in this article. Those times remain measurements of the historical implementation, identified by its retained source manifest. The candidate's `tools/differential_canonical.py` has a separate log with 72 comparisons against the three campaigns and no recorded mismatches under its stated error/count criteria. Its accepted-step sequences are outputs of the candidate run, not reconstructed historical sequences. This check neither establishes equal runtime nor reexecutes the 108-case fixed-step method campaign, which remains historical evidence. No new cluster timing of the private snapshot is claimed.

The cross-check companion provides the two drivers, 102 original/corrected scheme output rows, coefficient values, refinement output, correction diffs and selected original-file hashes. Supplementary Table S6 still reproduces the unchanged summary; the additional files make its test configuration and source boundaries inspectable. The supplied coverage index connects all 47 catalogue entries with source definitions and concrete test identifiers; it distinguishes structural, coefficient, scalar-order and linear tests from declared order metadata.

For manuscript reconstruction, `figure_sources/make_figures.py` regenerates the numerical panels and shared legend from the selected records. `tables/make_tables.py` generates four tables (startup, catalogue, Lawson and development step doubling), and `tables/make_crosscheck_table.py` generates Table S6 from the preserved summary. Other tables are supplied LaTeX compositions. The figure and table transformations are separate from integrator execution and from the recorded installation tests. An evidence index identifies the private archive and the paths of its logs without replacing historical identifiers.

9.3 AI-assisted research and manuscript workflow

Generative-AI tools were used during the research lifecycle as interactive assistants: ChatGPT (OpenAI) for proposing diagnostic experiments, drafting and checking shell and Python snippets, organizing software-release audits and assembling an earlier draft of this manuscript, and Claude (Anthropic, model Claude Fable 5.1) for the independent review of that draft, the correction blocks of the software, the cross-check against the original EXPINT sources and the revision of the text. The tools did not generate the numerical values reported as experimental results; those come from archived PyEXPINT executions and campaign records. Checks described in the manuscript include code execution, regression tests, comparison of preserved sources and algebraic deductions. These forms of evidence are identified separately; the algebraic ETD34 example is not represented as a new numerical execution, and execution logs are distinguished from AI-generated descriptions. The experimental scope, the scientific interpretation, the code admitted to the release and the claims of the manuscript are the responsibility of the human author. This section is included because the assistance covered experiment design, coding, analysis, validation and reproducibility, not only editing.

9.4 Threats to validity and claim boundaries

Wall time is local evidence. The canonical timing campaigns are replicated executions on a specified cluster environment, not a cross-machine performance study. Two of the three ran on the same node, and the between-replica spread of the paired ratios is several times their within-replica interquartile range. The paper uses timing ratios as a demonstration of the benchmarking infrastructure and relies on achieved error and work counters for portable interpretation.

The canonical problem is a single Fourier mode. Its manufactured solution is an eigenvector of the periodic operator, so the vectors acted upon are dominated by a few modes and the Krylov counts depend weakly on the grid in the measured cases. The campaign says nothing about operators with broadband data, which the problem of Section 7.6 addresses through the supplied original workstation record; half of the canonical campaign is maximum-step-limited.

A work counter is not total cost. A matrix-vector product in Arnoldi, Leja or a hybrid path has different associated vector and projected-matrix work, which is why the Leja backend costs 1.08 to 1.47 times the KIOPS-type time with 1.63 to 3.11 times its products. The hybrid counter omits the internal SciPy products. Estimated algorithmic workspace is not peak resident memory.

The public Leja path has no substepping. The real-Leja backend interpolates on the whole scaled spectral interval with a maximum degree of 80: degree 69 is used at width 240 on a dissipative operator and the cap is reached near width 300 at tolerance 10^{-10}; the canonical problem reaches width 53 and the broadband problem 210. Wider intervals require a larger degree or a smaller step, and the failure is reported as a rejected step by the adaptive solvers.

Tolerance semantics are not automatically equivalent. Temporal tolerances, matrix-action tolerances and stopping criteria play different roles across realizations. The canonical timing campaign records an action fraction of 0.01; the package default is 0.05. The work–precision experiments either fix the action backend and tolerance or display achieved error, rather than treating a requested tolerance as an accuracy identity.

The ETD34 estimator is blind to state-independent forcing. Section 6 documents that the published pair estimates no error for $N(t, y) = g(t)$, and the adaptive comparisons of Section 7.5 are read with that confound in mind.

Exact-history experiments isolate method bodies. The fixed-window method work–precision experiment uses exact nonlinear history before the common numerical interval. This isolates the body of multistep and general linear schemes; with the default numerical startup the guaranteed end-to-end order is $\min(p, 5)$ and the observed slopes for the selected order six and seven entries between $h = 0.05$ and $h = 0.025$ lie between 4.55 and 4.80 (Section 7.7).

Automatic selection is heuristic. The rule-based selector uses operator structure, problem size and a scaled spectral-width threshold. On the canonical problem it chose the slower backend in every configuration. Its thresholds are exposed, a calibrated selector interface exists, and the current evidence does not support a globally optimal selector.

No external-speed or distributed-scaling claim. The relationship to other packages is established functionally in Section 8 and, for the historical catalogue, by execution of the original EXPINT sources; no controlled runtime comparison against external packages is part of this article, and the release has no distributed backend contract.

10 Conclusions

PyEXPINT demonstrates that a broad historical exponential-integrator catalogue can be represented as mathematical data and executed through a small number of well-defined software boundaries. The 47 reconstructed entries share one explicit exponential general linear engine; analytic φ expressions are separated from their numerical realization; history and startup are explicit; and work instrumentation is collected at the backend boundary. This architecture makes dense, diagonal, Krylov, KIOPS-type, real-Leja, hybrid and automatic realizations available without duplicating the time integrators themselves, and admits external realizations through one hook.

The validation work is as important as the abstraction. The supplied comparison summary reports near-roundoff agreement with the original EXPINT implementation for 42 entries, agreement after one-line corrections for four entries, and regenerated output coefficients for one entry. Its aggregate and named results, comparison drivers and detailed original outputs are now archived with the identified private software companion. Coefficient and metadata inconsistencies are reconciled by structural and algebraic diagnostics, confirmed by execution and guarded by regressions. The fusion ablation shows a concrete benefit of representing action algebra explicitly, with both Krylov realizations, while the work–precision and timing experiments show why performance should not be reduced to a single counter or backend ranking. The canonical paired timing experiment favours the KIOPS-type backend for every tested configuration, but its manufactured solution favours Krylov realizations by construction and the rule-based selector chooses the slower backend throughout; the appropriate conclusion is problem- and environment-dependent selection, not a universal winner.

The embedded ETD34 solver shows how the action boundary serves an adaptive integrator: matrix-action tolerance is distinct from temporal tolerance, failed iterative actions and non-finite trials become rejected steps, termination is explicit, and endpoint nonlinear values are reused. It also inherits a limitation of the published pair, an error estimate that vanishes for state-independent forcing, which is documented rather than hidden, and the default numerical startup does not in general deliver the nominal orders of the high-order history methods. The original broadband record shows a change in the faster realization at closely comparable accuracy for selected tolerance pairs. Its workstation environment, initial/reference states and individual timings are preserved; the least accurate setting is not a matched-error speed comparison.

The manuscript package makes the supplied numerical summaries and their postprocessing inspectable. The identified private candidate, validation and installation logs, original experiment records and a family-to-test evidence map accompany it. The claims distinguish recorded software

execution, independently checkable data transformations and limitations that remain outside those tests. The next technical directions are outside the present claims: variable-step history methods, order-matched numerical startup, robust embedded pairs, data-driven backend selection, cost models that include orthogonalization and nonlinear work, and communication-aware distributed action backends. The catalogue and the common execution layer provide a controlled platform on which those extensions can be studied without reopening the mathematical definition of every method.

Acknowledgments

This work was supported by the Universidad de Málaga through project PPRO-B4-2026-004 of its Plan Propio de Investigación, Transferencia y Divulgación Científica. The computations of the canonical timing campaigns were carried out on the Picasso supercomputer of the Supercomputing and Bioinnovation Center (SCBI) of the Universidad de Málaga. The use of AI-assisted tools is described in Section 9.3.

References

[1] Awad H. Al-Mohy and Nicholas J. Higham. 2011. Computing the Action of the Matrix Exponential, with an Application to Exponential Integrators. *SIAM Journal on Scientific Computing* 33, 2 (2011), 488–511. doi:10.1137/100788860

[2] Håvard Berland, Brynjulf Owren, and Bård Skaflestad. 2005. B-Series and Order Conditions for Exponential Integrators. *SIAM J. Numer. Anal.* 43, 4 (2005), 1715–1727. doi:10.1137/040612683

[3] Håvard Berland, Bård Skaflestad, and Will M. Wright. 2007. EXPINT—A MATLAB Package for Exponential Integrators. *ACM Trans. Math. Software* 33, 1, Article 4 (2007), 17 pages. doi:10.1145/1206040.1206044

[4] Marco Caliari, Peter Kandolf, Alexander Ostermann, and Stefan Rainer. 2016. The Leja Method Revisited: Backward Error Analysis for the Matrix Exponential. *SIAM Journal on Scientific Computing* 38, 3 (2016), A1639–A1661. doi:10.1137/15M1027620

[5] Marco Caliari, Marco Vianello, and Luca Bergamaschi. 2004. Interpolating Discrete Advection–Diffusion Propagators at Leja Sequences. *J. Comput. Appl. Math.* 172, 1 (2004), 79–99. doi:10.1016/j.cam.2003.11.015

[6] S. M. Cox and P. C. Matthews. 2002. Exponential Time Differencing for Stiff Systems. *J. Comput. Phys.* 176, 2 (2002), 430–455. doi:10.1006/jcph.2002.6995

[7] Pranab J. Deka, Lukas Einkemmer, and Mayya Tokman. 2023. LeXInt: Package for Exponential Integrators Employing Leja Interpolation. *SoftwareX* 21 (2023), 101302. doi:10.1016/j.softx.2022.101302

[8] Pranab J. Deka, Alexander Moriggl, and Lukas Einkemmer. 2023. LeXInt: GPU-Accelerated Exponential Integrators Package. arXiv:2310.08344 [math.NA]

[9] Stéphane Gaudreault, Greg Rainwater, and Mayya Tokman. 2018. KIOPS: A Fast Adaptive Krylov Subspace Solver for Exponential Integrators. *J. Comput. Phys.* 372 (2018), 236–255. doi:10.1016/j.jcp.2018.06.026

[10] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array Programming with NumPy. *Nature* 585 (2020), 357–362. doi:10.1038/s41586-020-2649-2

[11] Marlis Hochbruck and Alexander Ostermann. 2005. Explicit Exponential Runge–Kutta Methods for Semilinear Parabolic Problems. *SIAM J. Numer. Anal.* 43, 3 (2005), 1069–1090. doi:10.1137/040611434

[12] Marlis Hochbruck and Alexander Ostermann. 2010. Exponential Integrators. *Acta Numerica* 19 (2010), 209–286. doi:10.1017/S0962492910000048

[13] Marlis Hochbruck and Alexander Ostermann. 2011. Exponential Multistep Methods of Adams-Type. *BIT Numerical Mathematics* 51, 4 (2011), 889–908. doi:10.1007/s10543-011-0332-6

[14] Georg Jansing. 2014. EXPODE—Advanced Exponential Time Integration Toolbox for MATLAB. arXiv:1404.4580. arXiv:1404.4580 [math.NA]

[15] Stein Krogstad. 2005. Generalized Integrating Factor Methods for Stiff PDEs. *J. Comput. Phys.* 203, 1 (2005), 72–88. doi:10.1016/j.jcp.2004.08.006

[16] J. Douglas Lawson. 1967. Generalized Runge–Kutta Processes for Stable Systems with Large Lipschitz Constants. *SIAM J. Numer. Anal.* 4, 3 (1967), 372–380. doi:10.1137/0704033

[17] Jitse Niesen and Will M. Wright. 2012. Algorithm 919: A Krylov Subspace Algorithm for Evaluating the φ-Functions Appearing in Exponential Integrators. *ACM Trans. Math. Software* 38, 3, Article 22 (2012), 19 pages. doi:10.1145/2168

773.2168781

[18] Alexander Ostermann, Mechthild Thalhammer, and Will M. Wright. 2006. A Class of Explicit Exponential General Linear Methods. *BIT Numerical Mathematics* 46 (2006), 409–431. doi:10.1007/s10543-006-0054-3

[19] PyEXPINT developers. 2026. PyEXPINT: Backend-Independent Exponential Integrators in Python. doi:10.5281/zenodo.22820595 Version 0.9.0; concept DOI 10.5281/zenodo.22820009.

[20] SciML contributors. [n. d.]. OrdinaryDiffEq.jl: Exponential Runge–Kutta Methods. Software documentation. https://docs.sciml.ai/OrdinaryDiffEq/stable/semilinear/ExponentialRK/ Accessed October 2026.

[21] SciML contributors. 2026. ExponentialUtilities.jl: Utilities for Matrix Exponentials and φ-Functions. Software repository. https://github.com/SciML/ExponentialUtilities.jl Accessed 2026-09-25.

[22] Roger B. Sidje. 1998. Expokit: A Software Package for Computing Matrix Exponentials. *ACM Trans. Math. Software* 24, 1 (1998), 130–156. doi:10.1145/285861.285868

[23] Bård Skaflestad and Will M. Wright. 2009. The Scaling and Modified Squaring Method for Matrix Functions Related to the Exponential. *Applied Numerical Mathematics* 59, 3–4 (2009), 783–799. doi:10.1016/j.apnum.2008.03.035

[24] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. doi:10.1038/s41592-019-0686-2

[25] Patrick Whalen. [n. d.]. rkstiff: Runge–Kutta Adaptive-Step Solvers for Stiff PDEs, Version 1.0.2. Python package. https://pypi.org/project/rkstiff/ Accessed October 2026.

[26] P. Whalen, M. Brio, and J. V. Moloney. 2015. Exponential Time-Differencing with Embedded Runge–Kutta Adaptive Step Control. *J. Comput. Phys.* 280 (2015), 579–601. doi:10.1016/j.jcp.2014.09.038

[27] Thoma Zoto and John C. Bowman. 2023. Robust Exponential Runge–Kutta Embedded Pairs. arXiv:2303.12139 [physics.comp-ph]